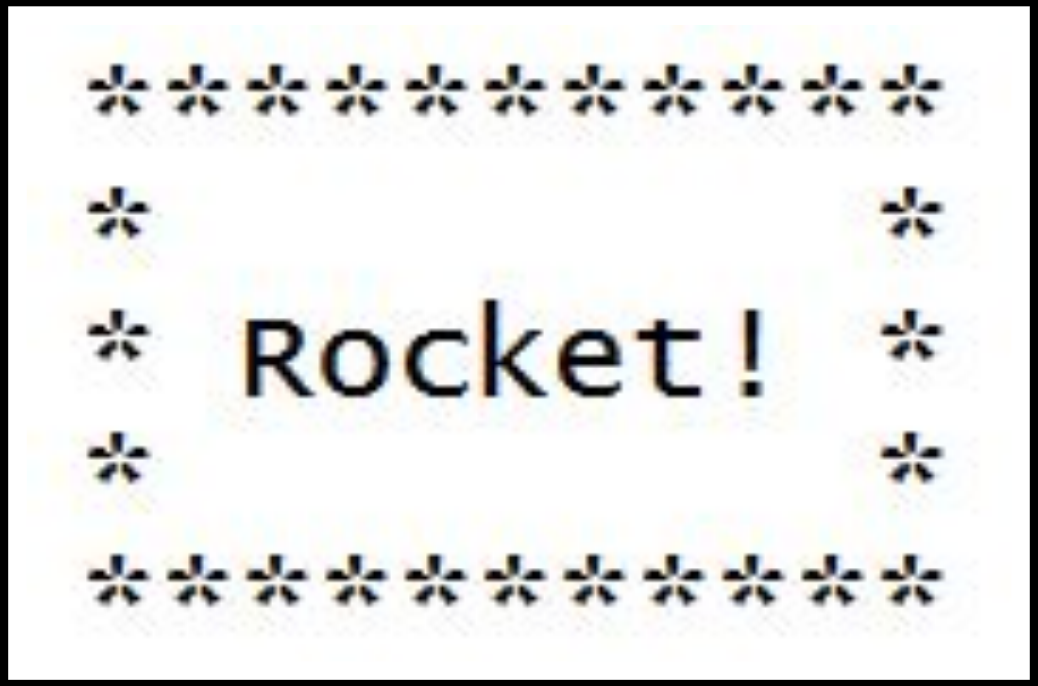
Strings And Files
ISTA 130

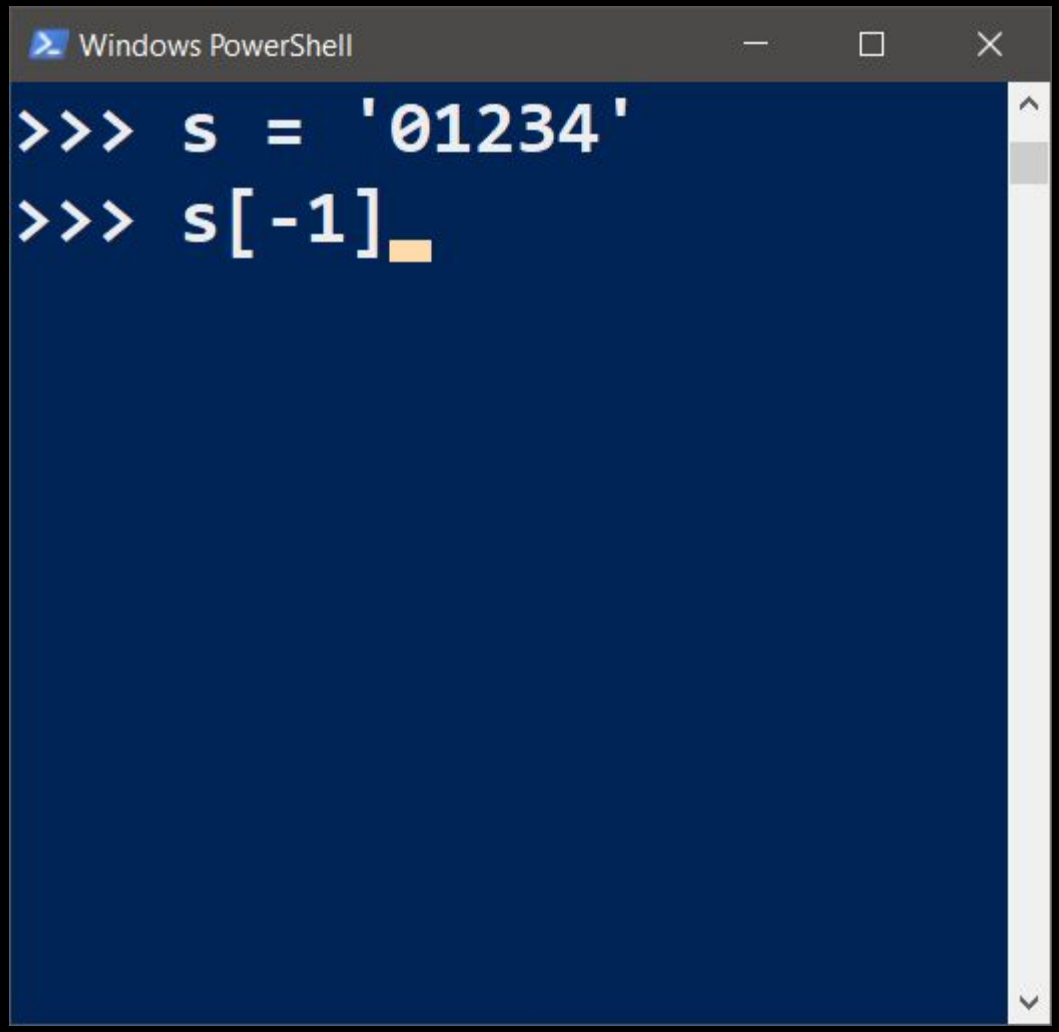
New Material on Strings the Week Before Midterm

- **Negative indexing**
- **`range` with multiple arguments**
- **Traverse by element**
- **Slicing to get substrings**
- **String methods**

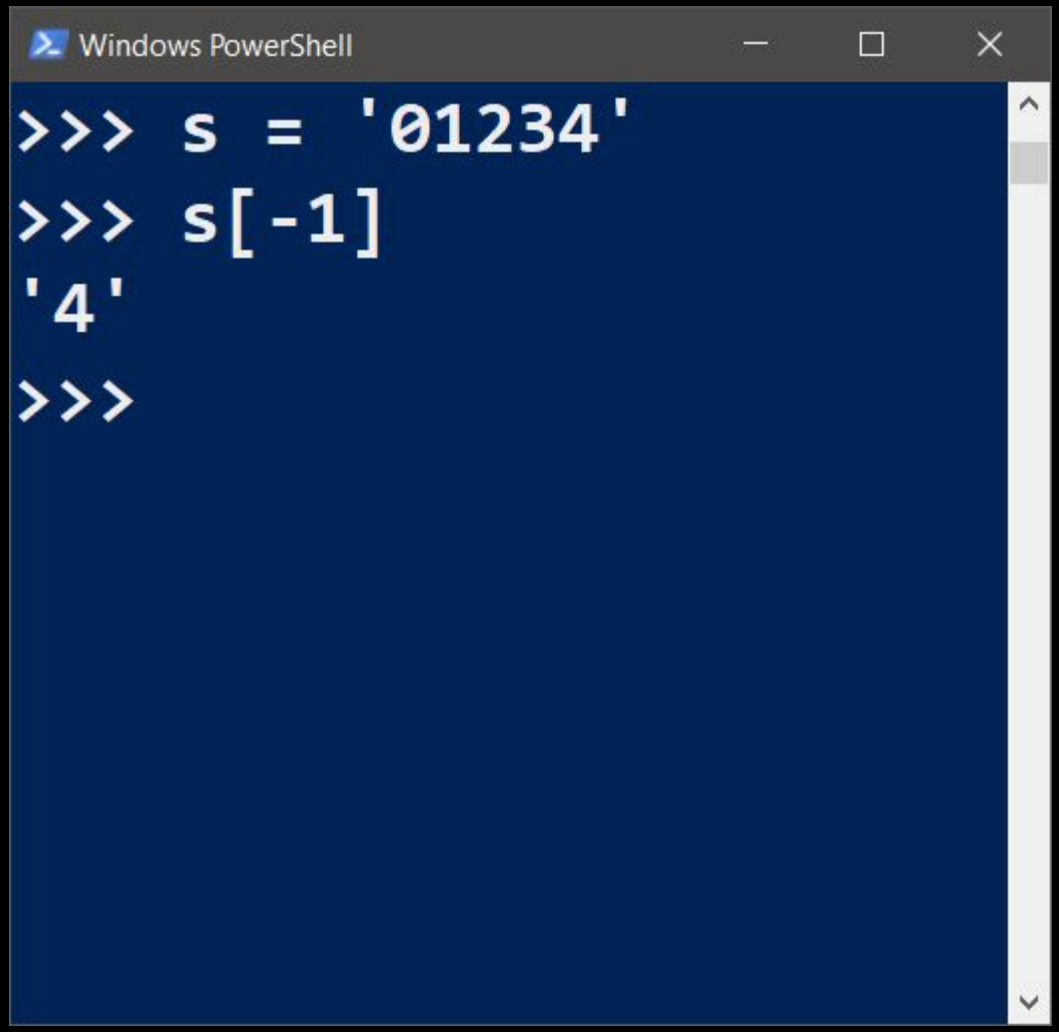


- Function `decorate_word`
- Takes one string argument
- Returns a new string that when printed looks like this – surrounded by a fancy box

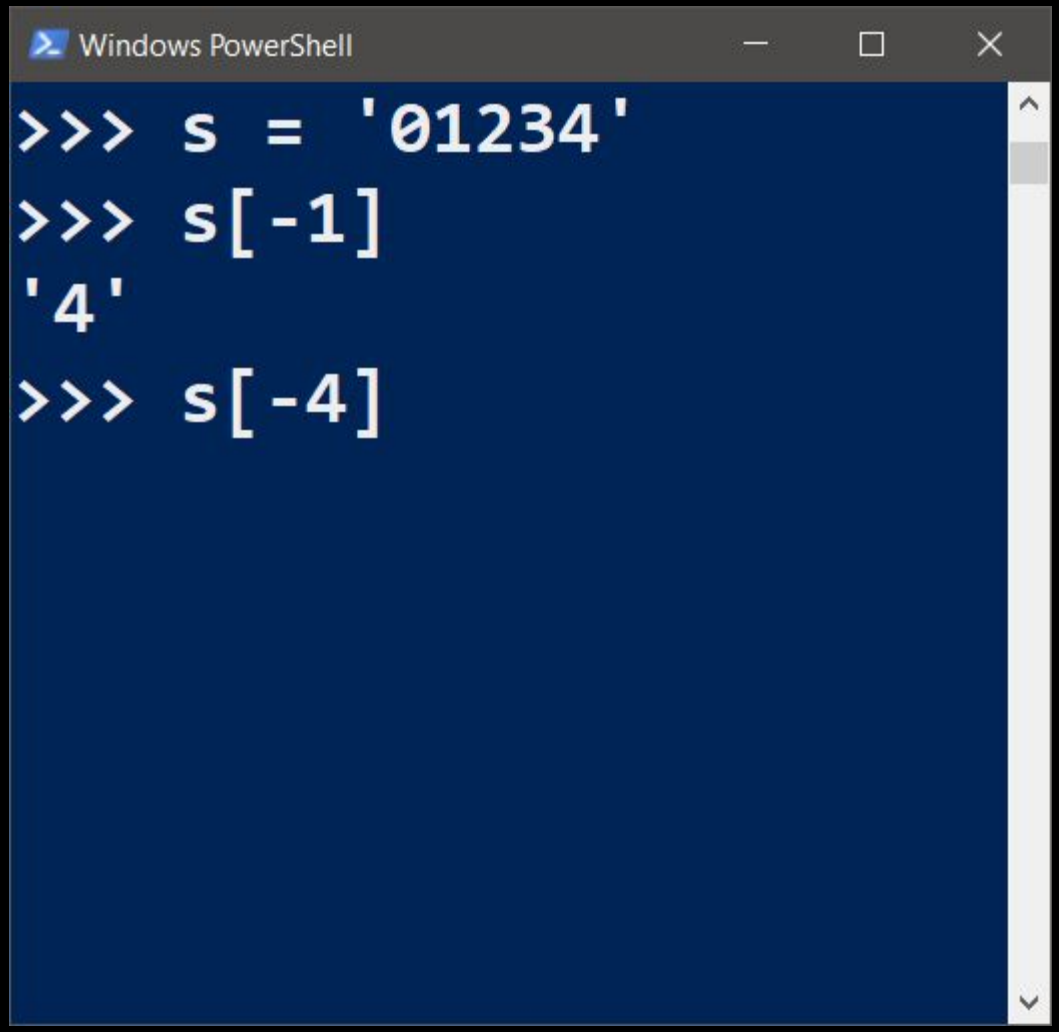
Negative Indexing



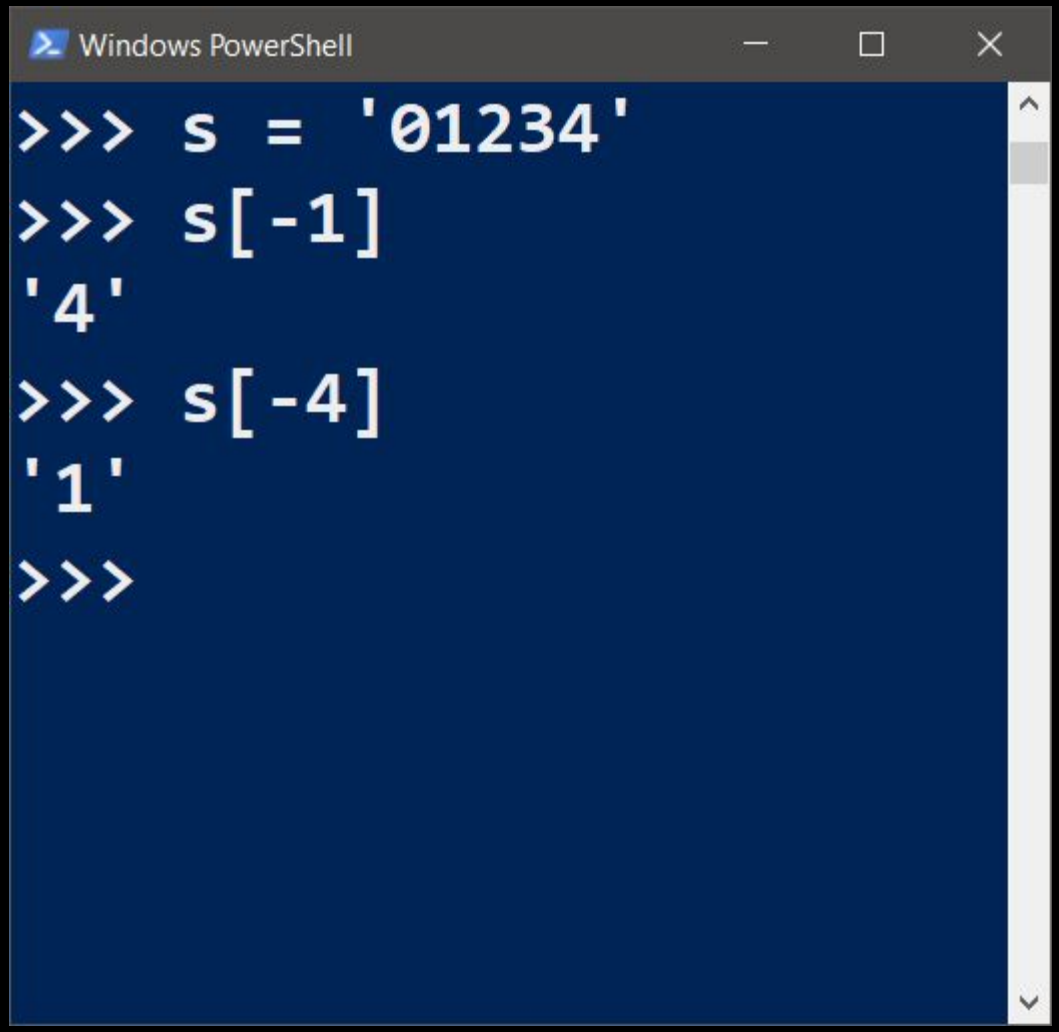
Negative Indexing



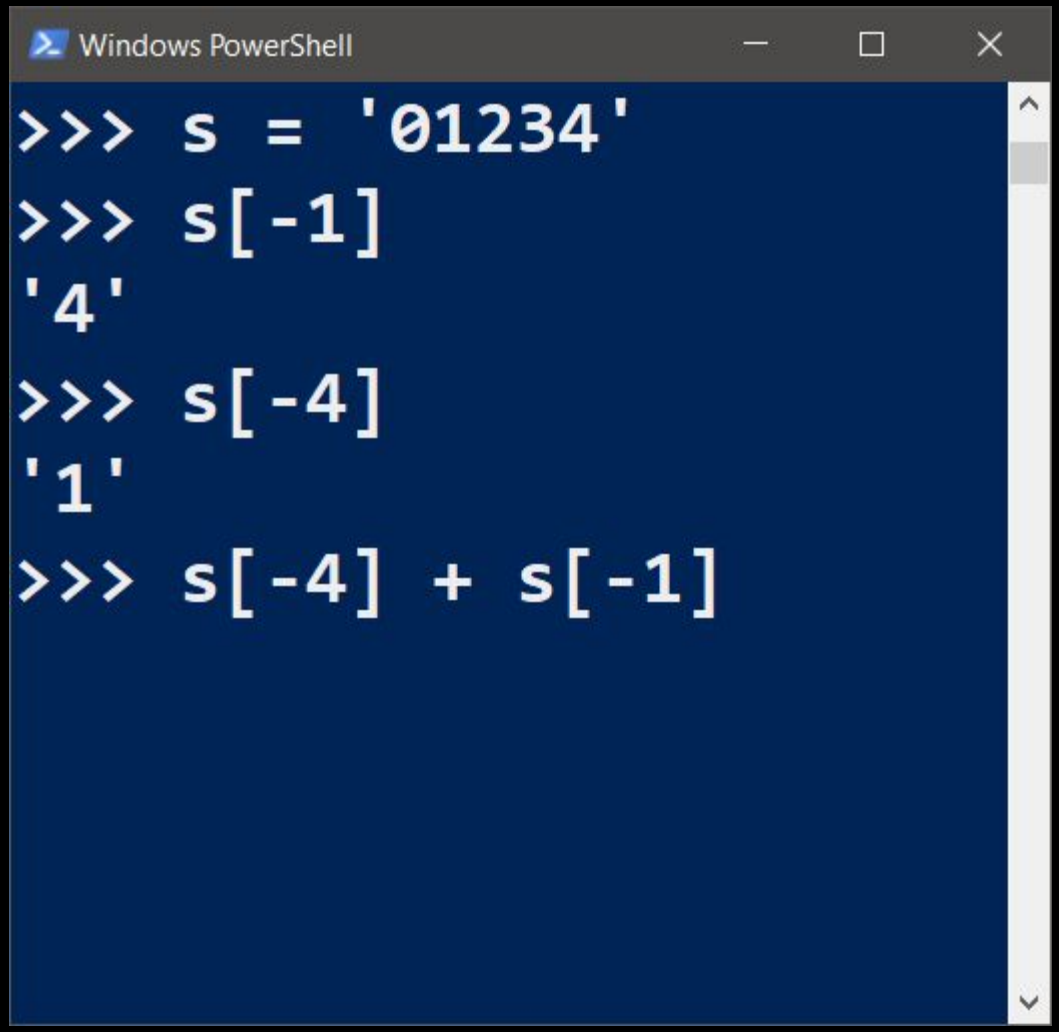
Negative Indexing



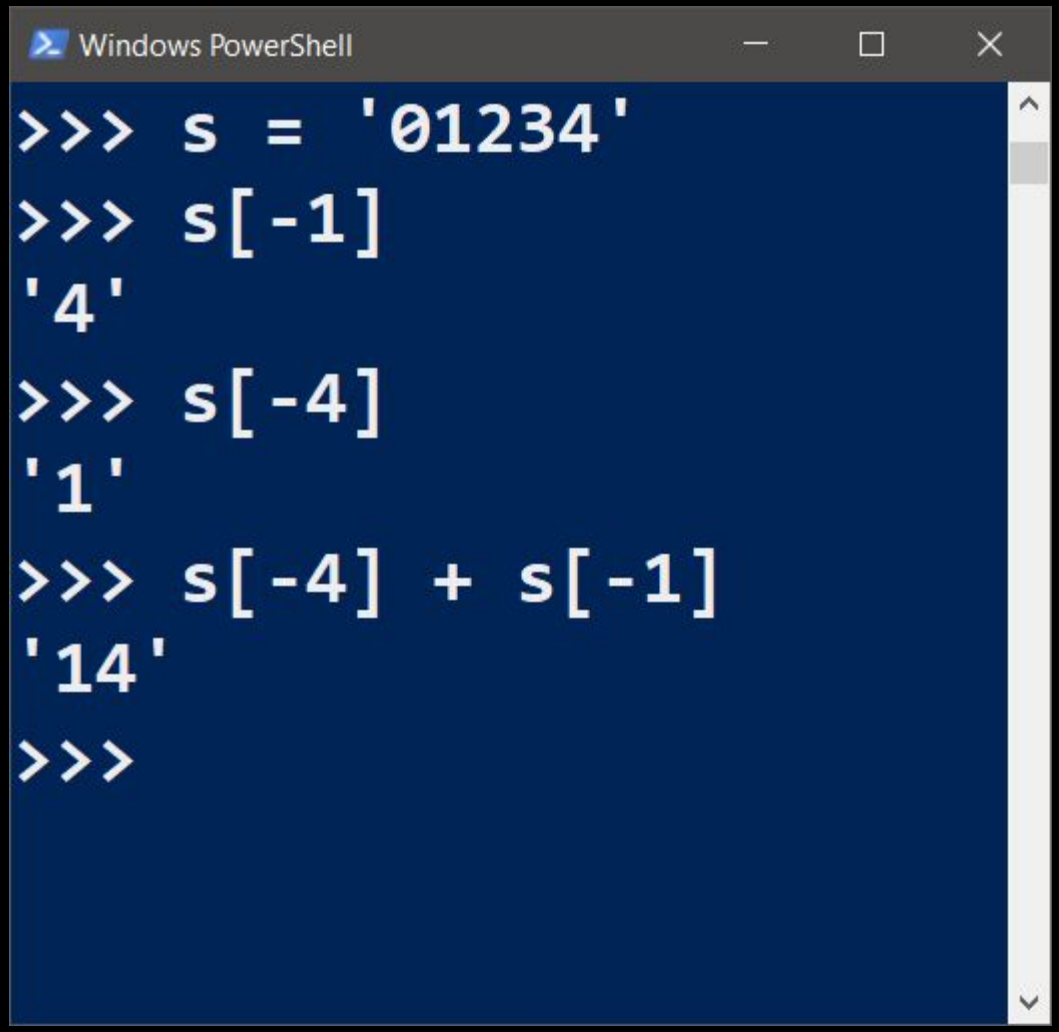
Negative Indexing



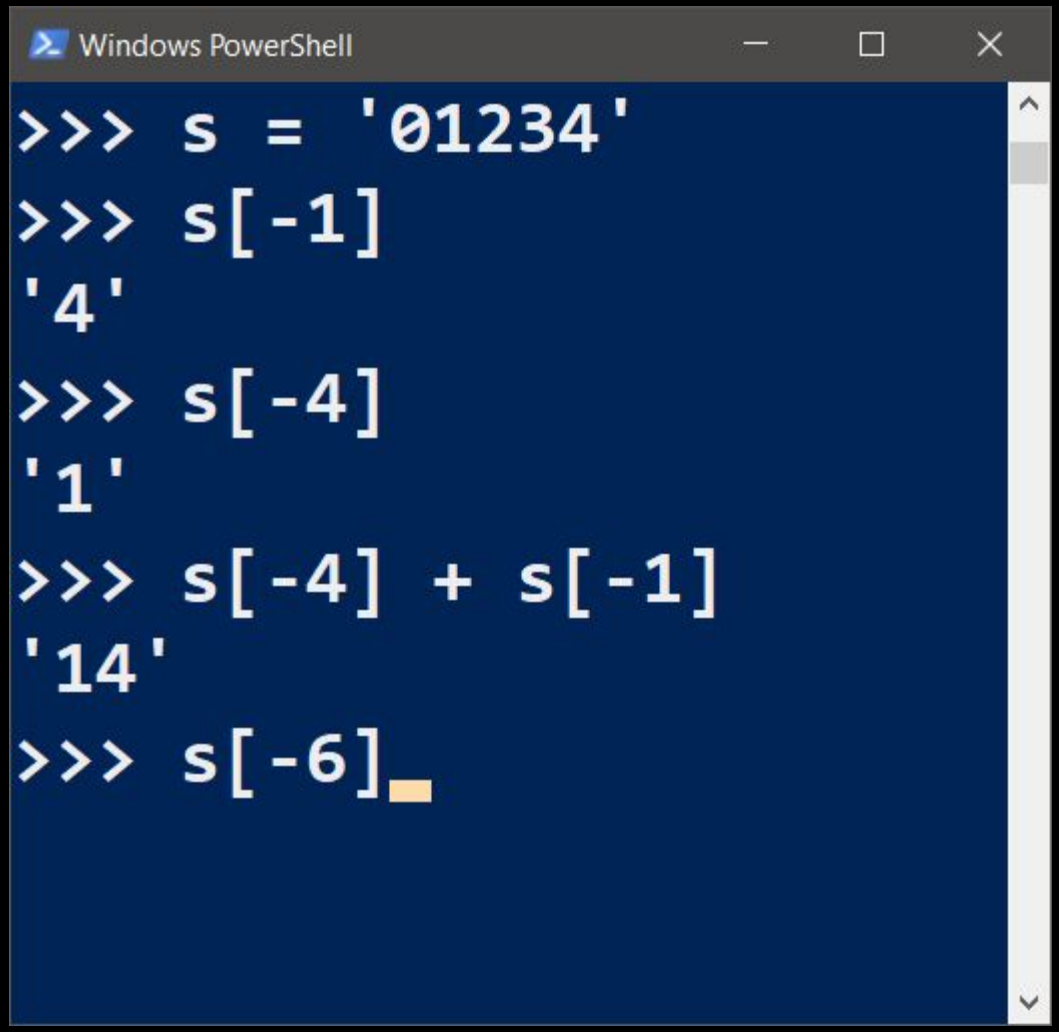
Negative Indexing



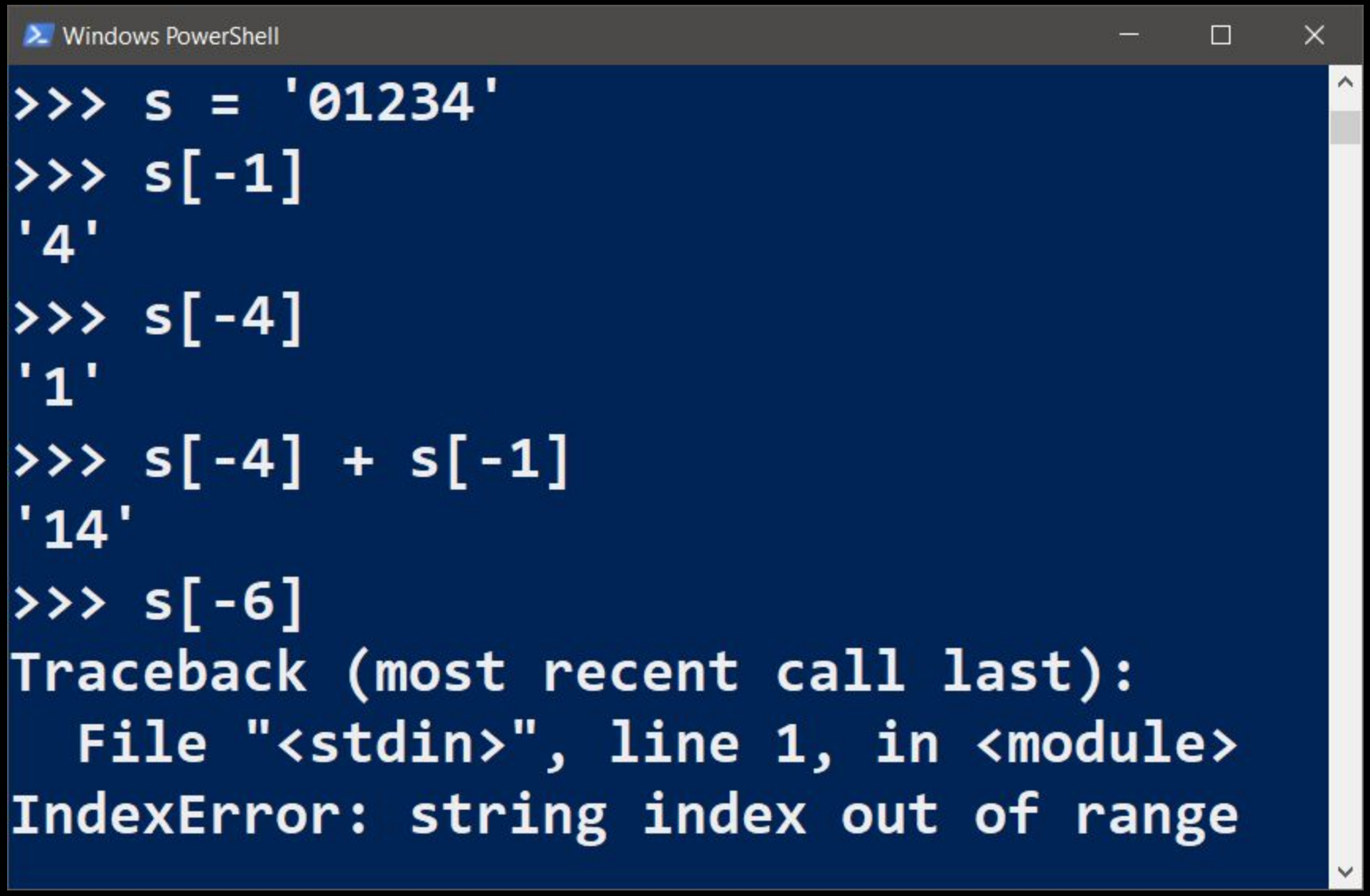
Negative Indexing



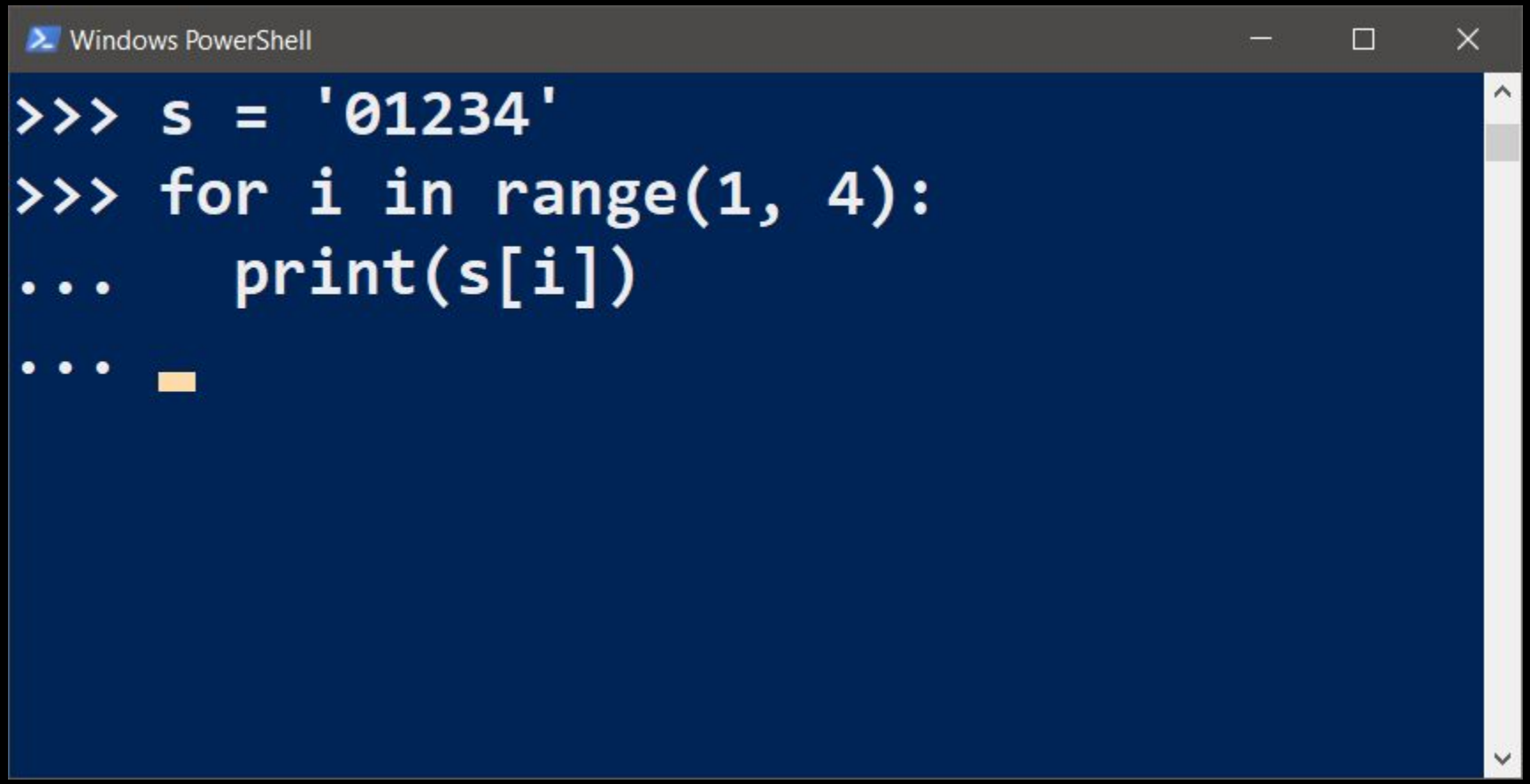
Negative Indexing



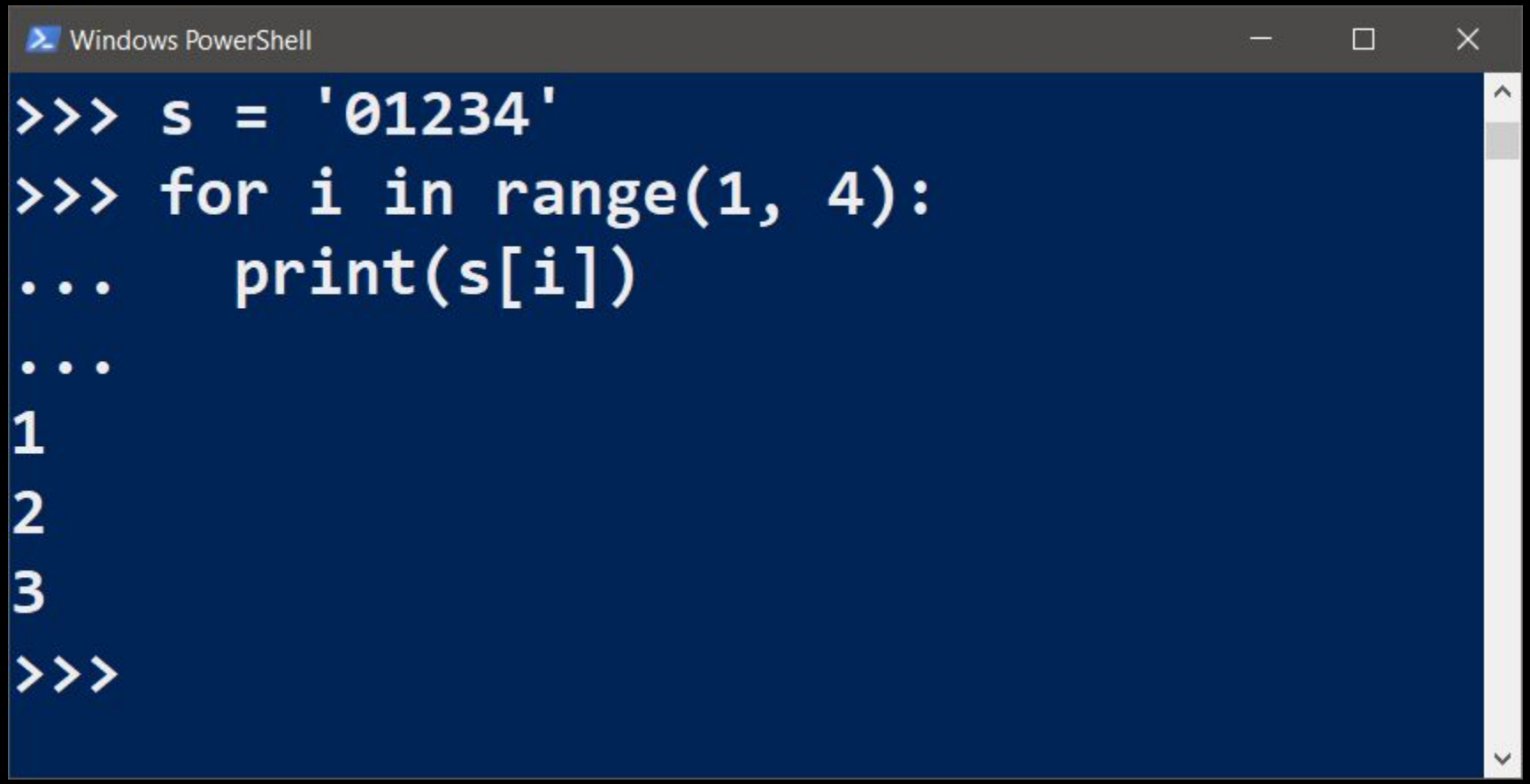
Negative Indexing



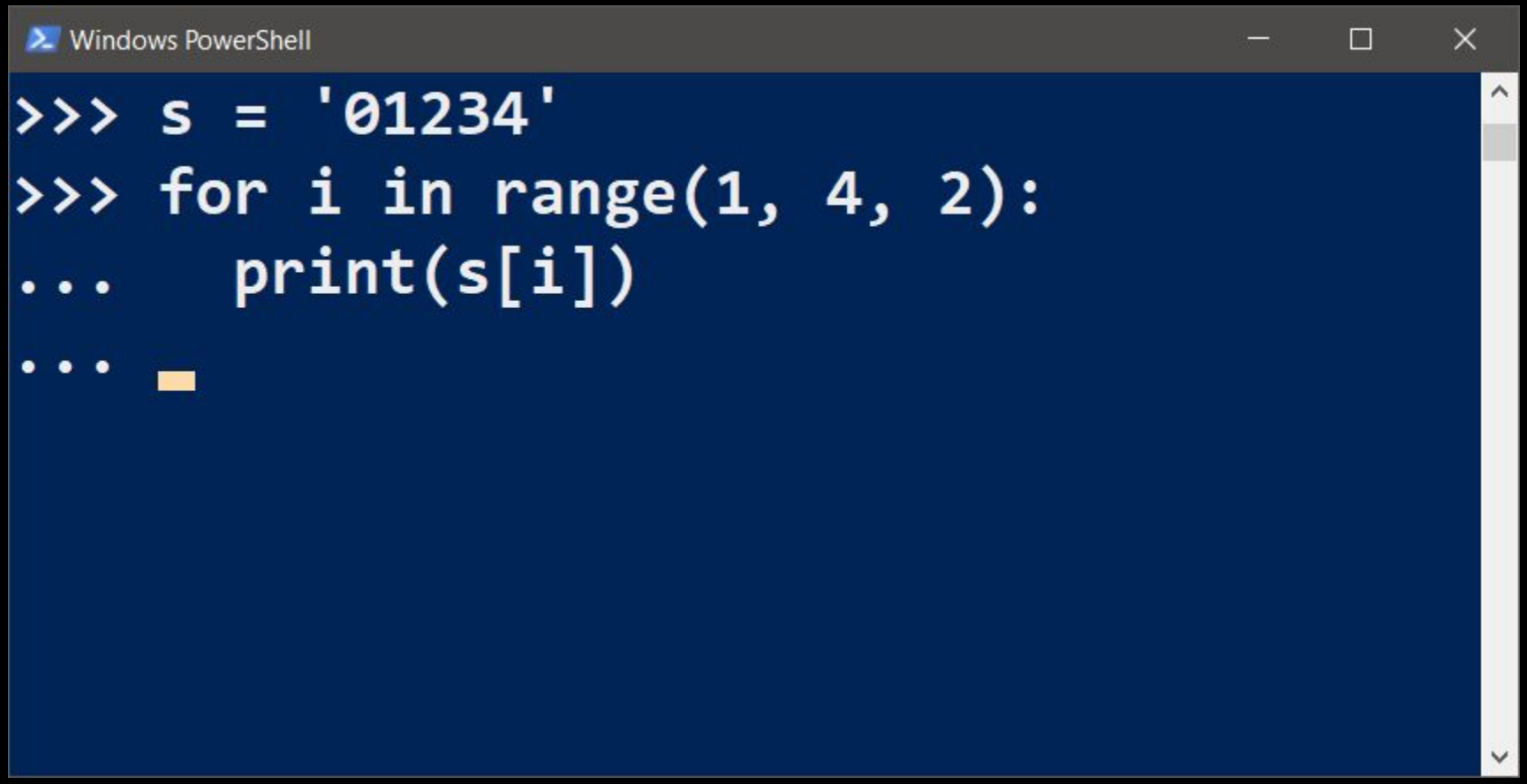
range With Multiple Arguments



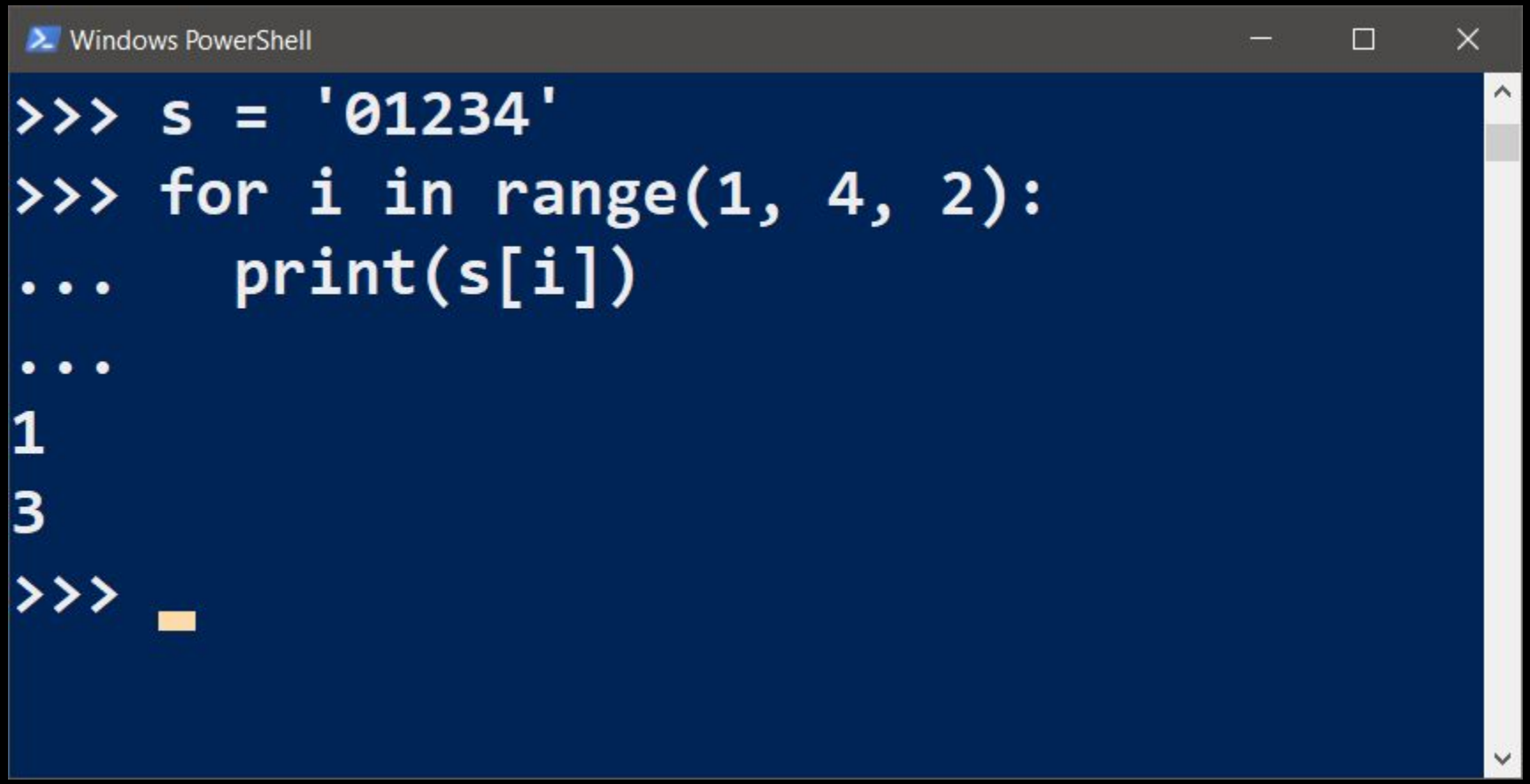
range With Multiple Arguments



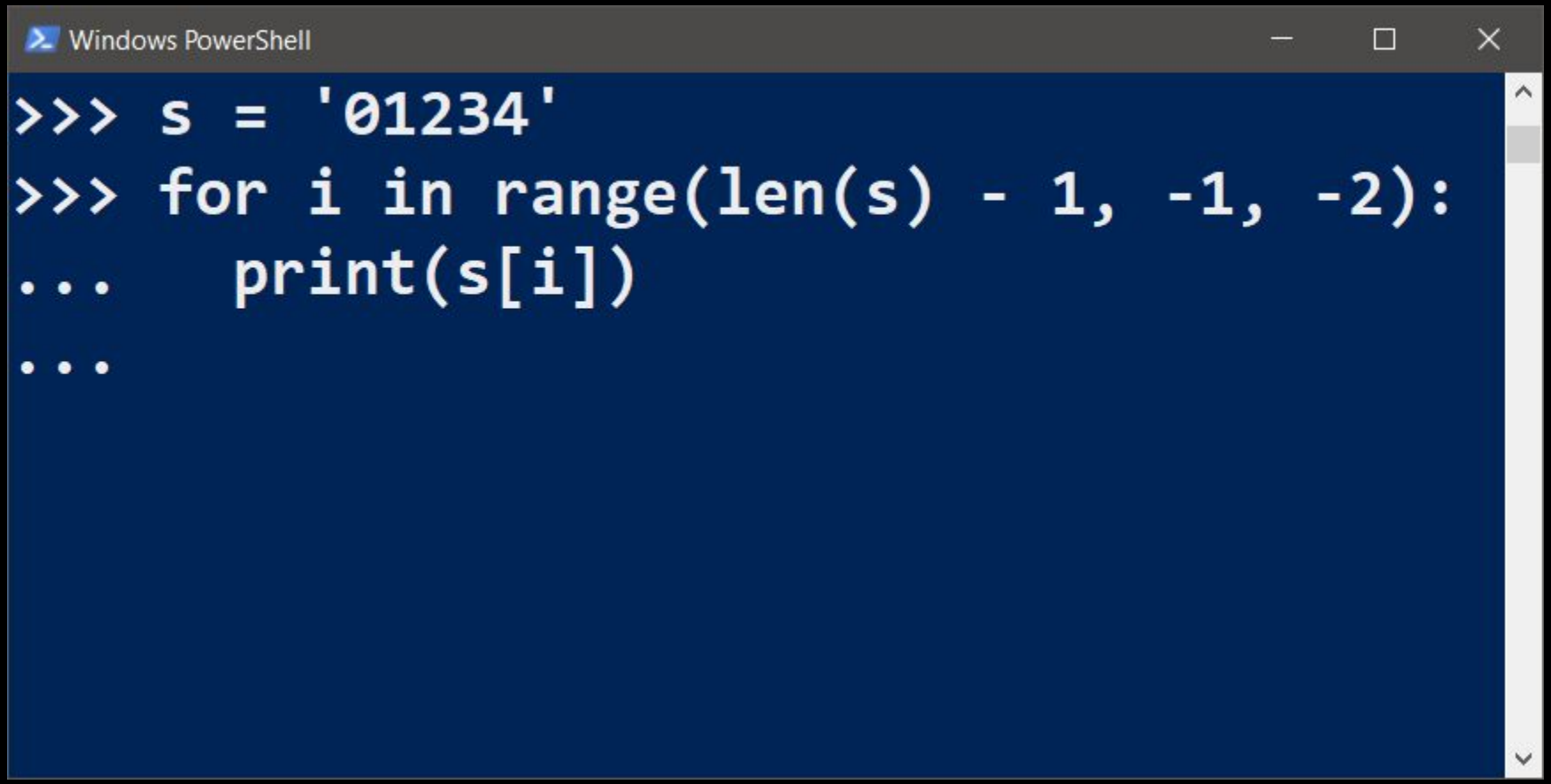
range With Multiple Arguments



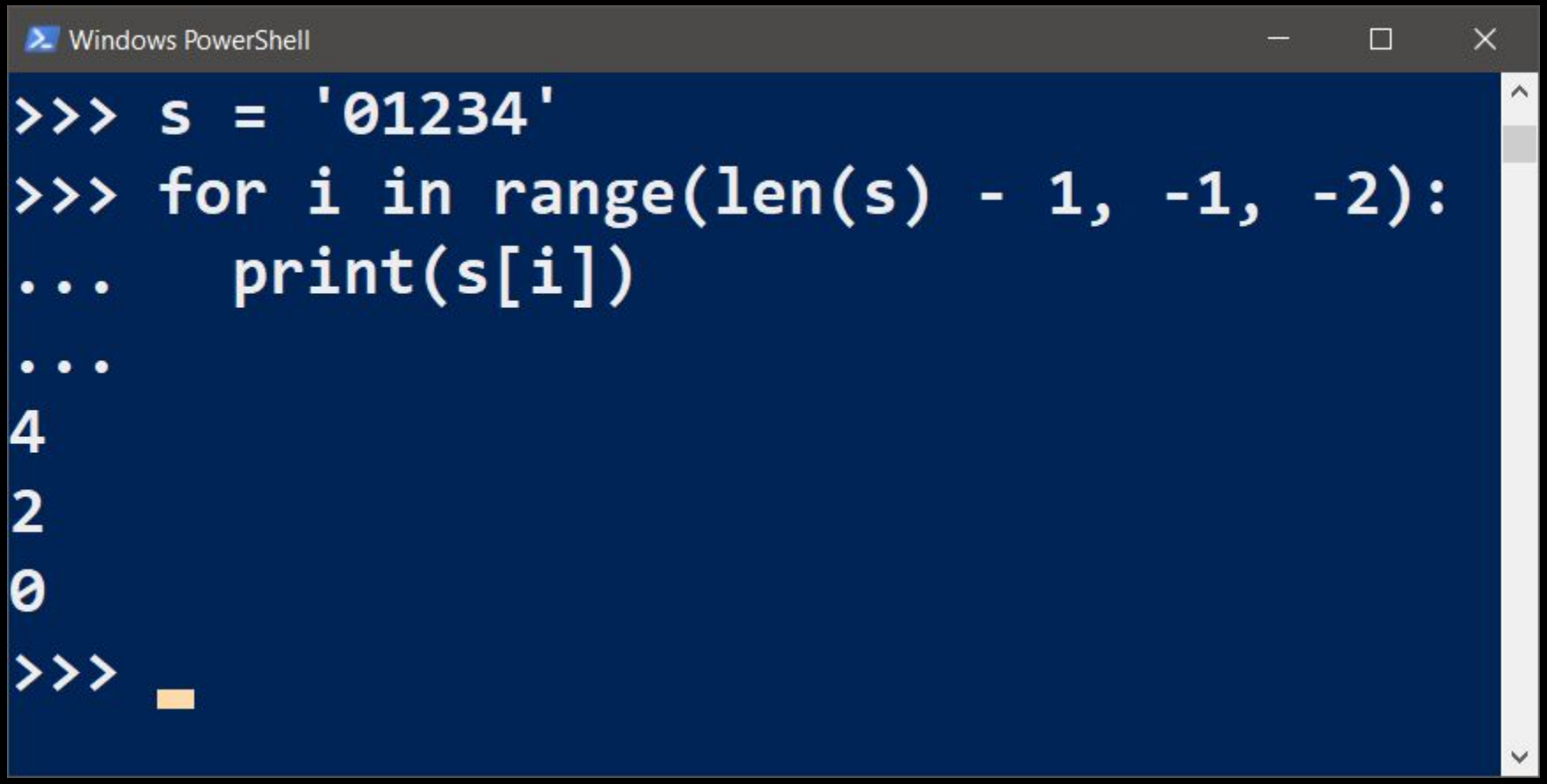
range With Multiple Arguments



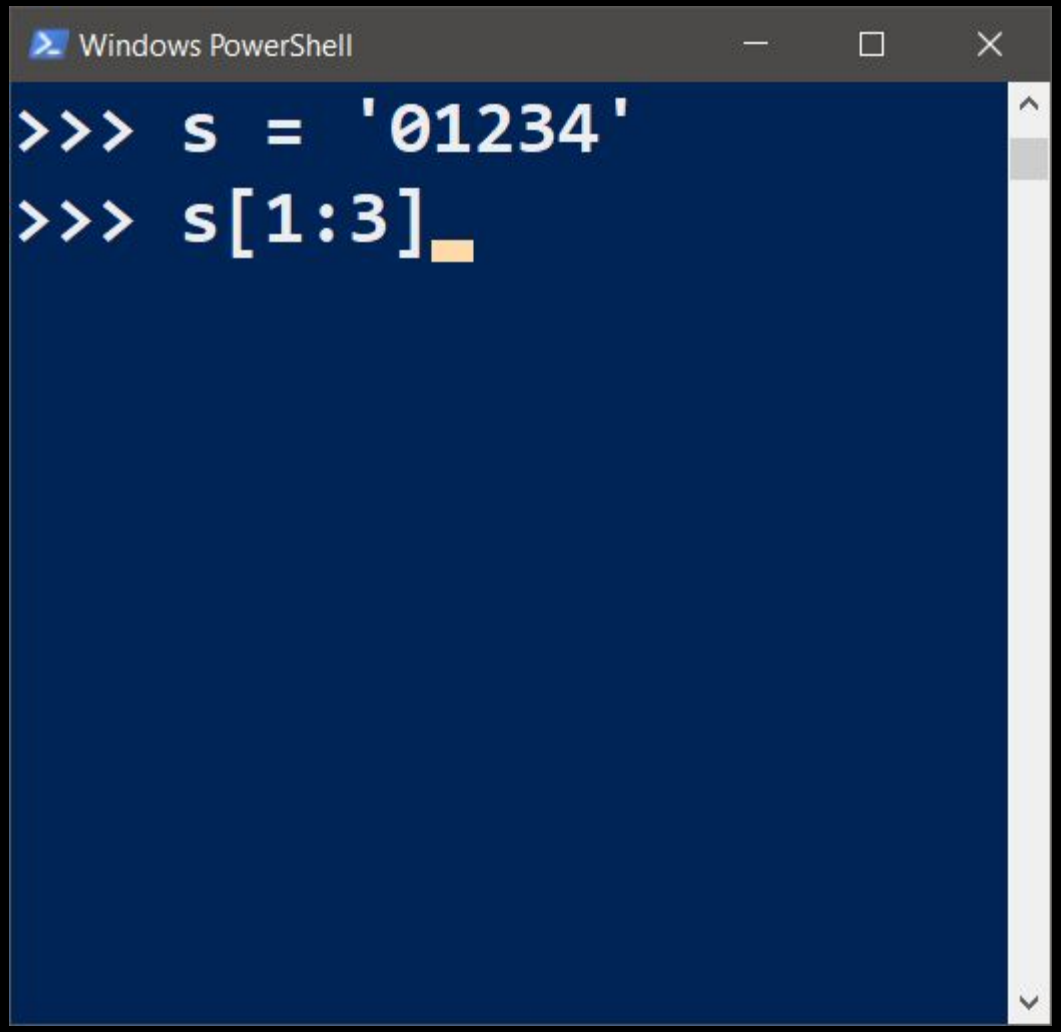
range With Multiple Arguments



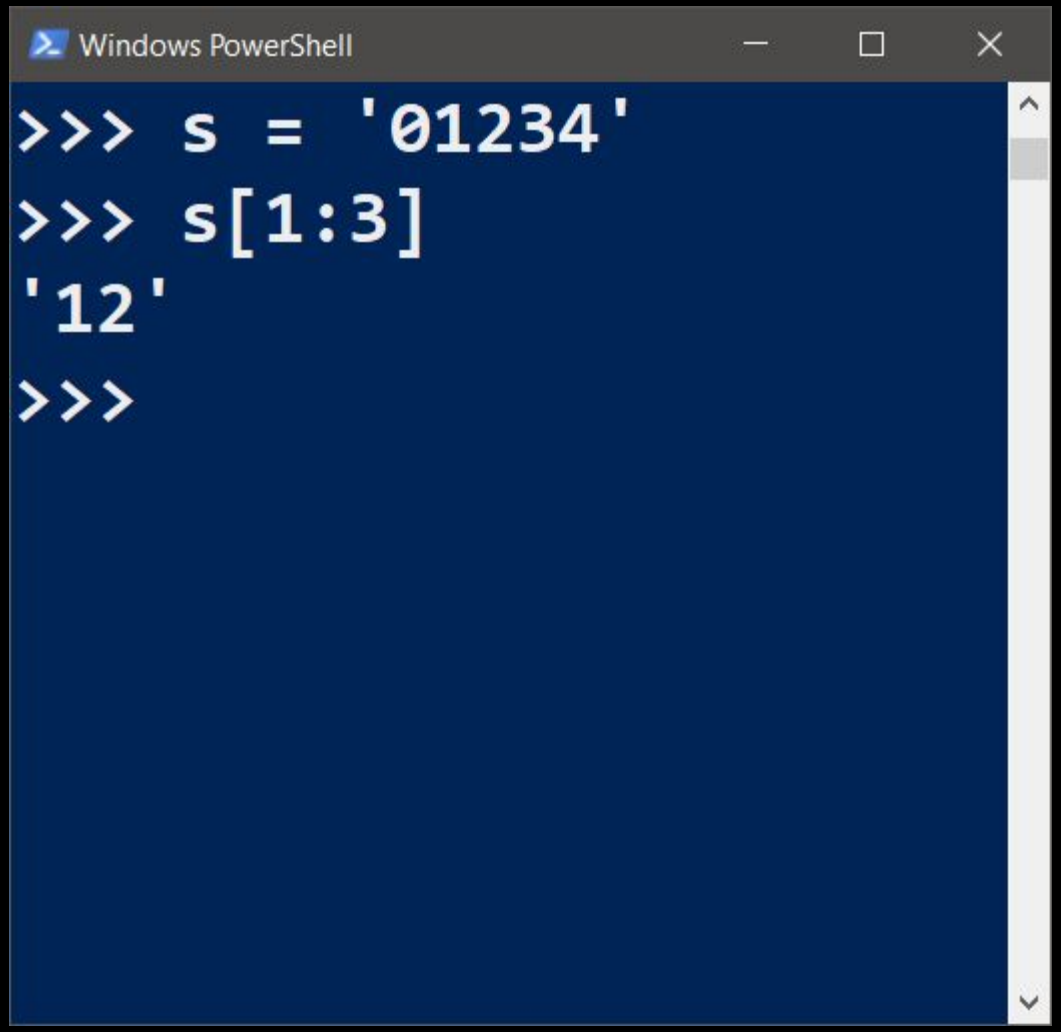
range With Multiple Arguments



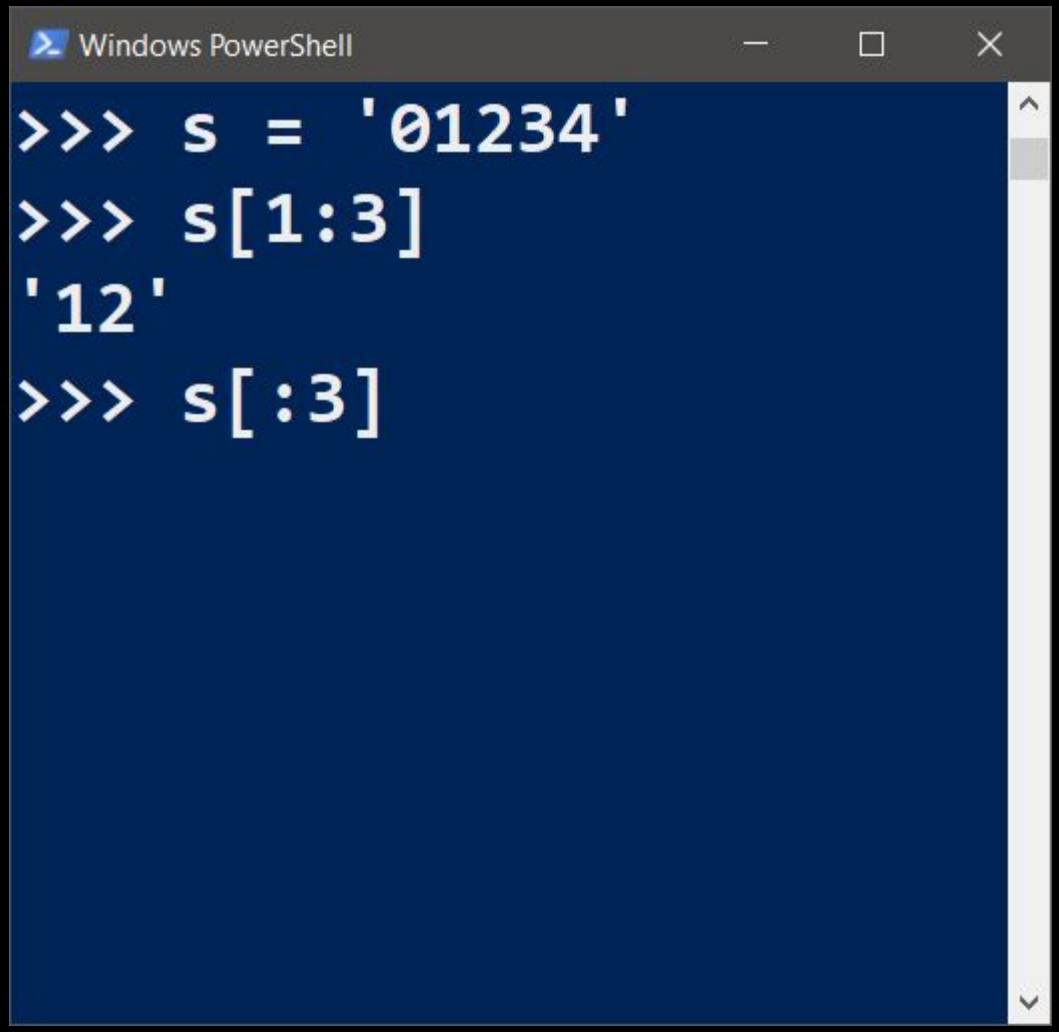
Some Slices



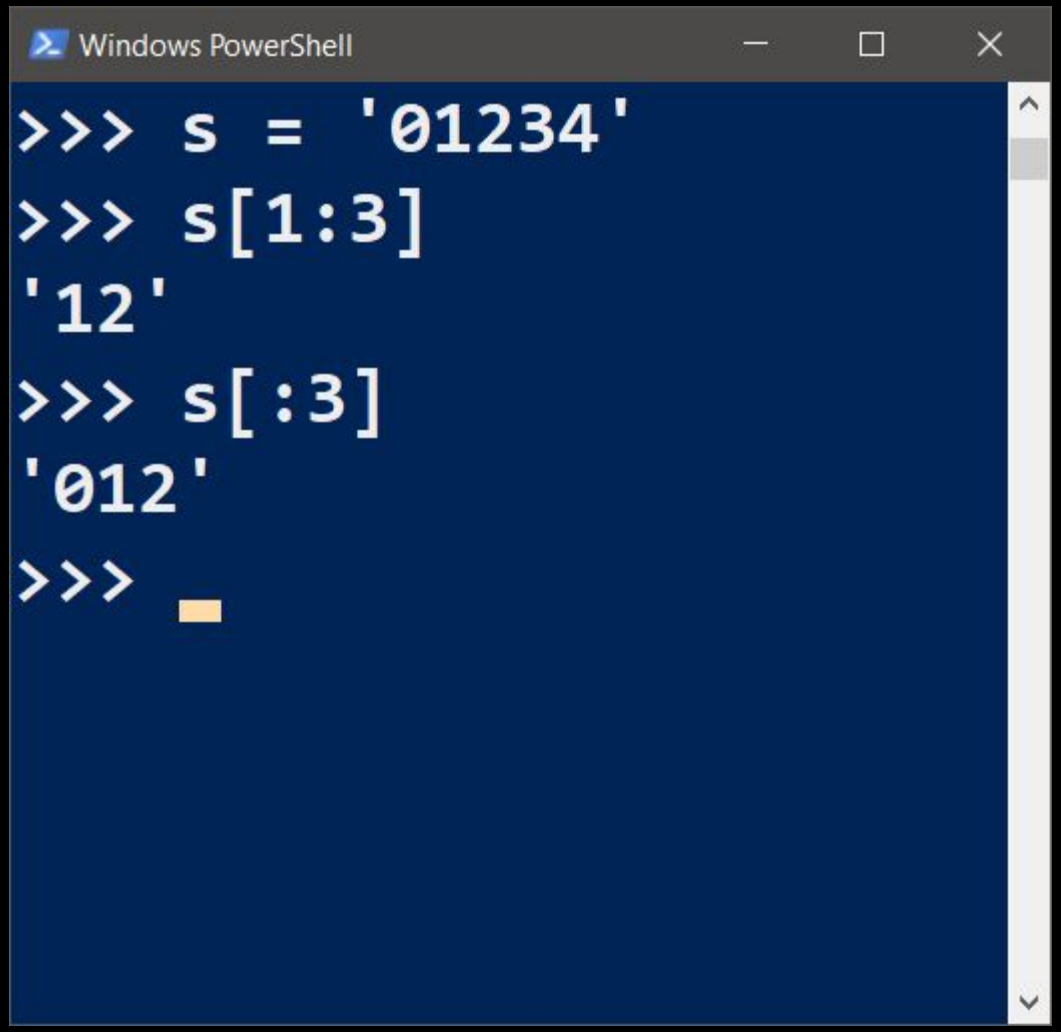
Some Slices



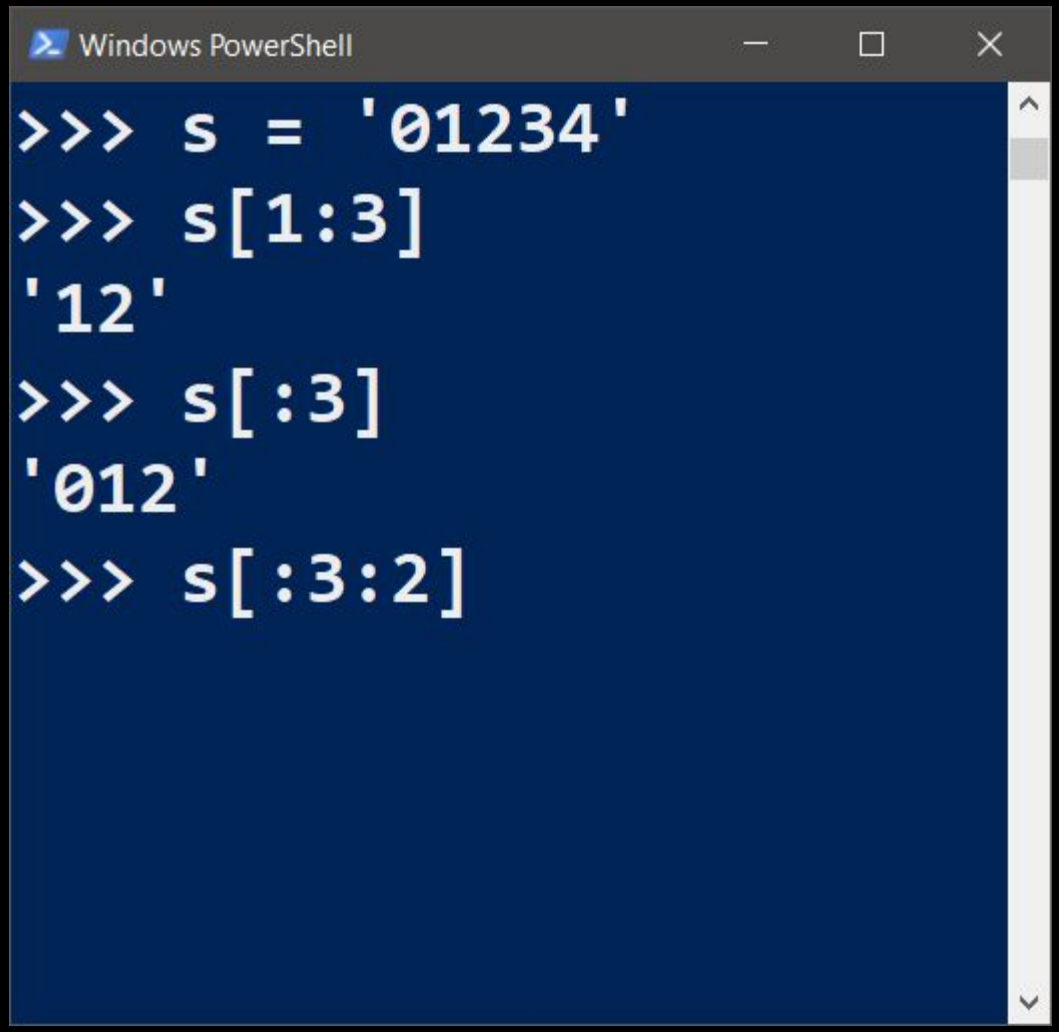
Some Slices



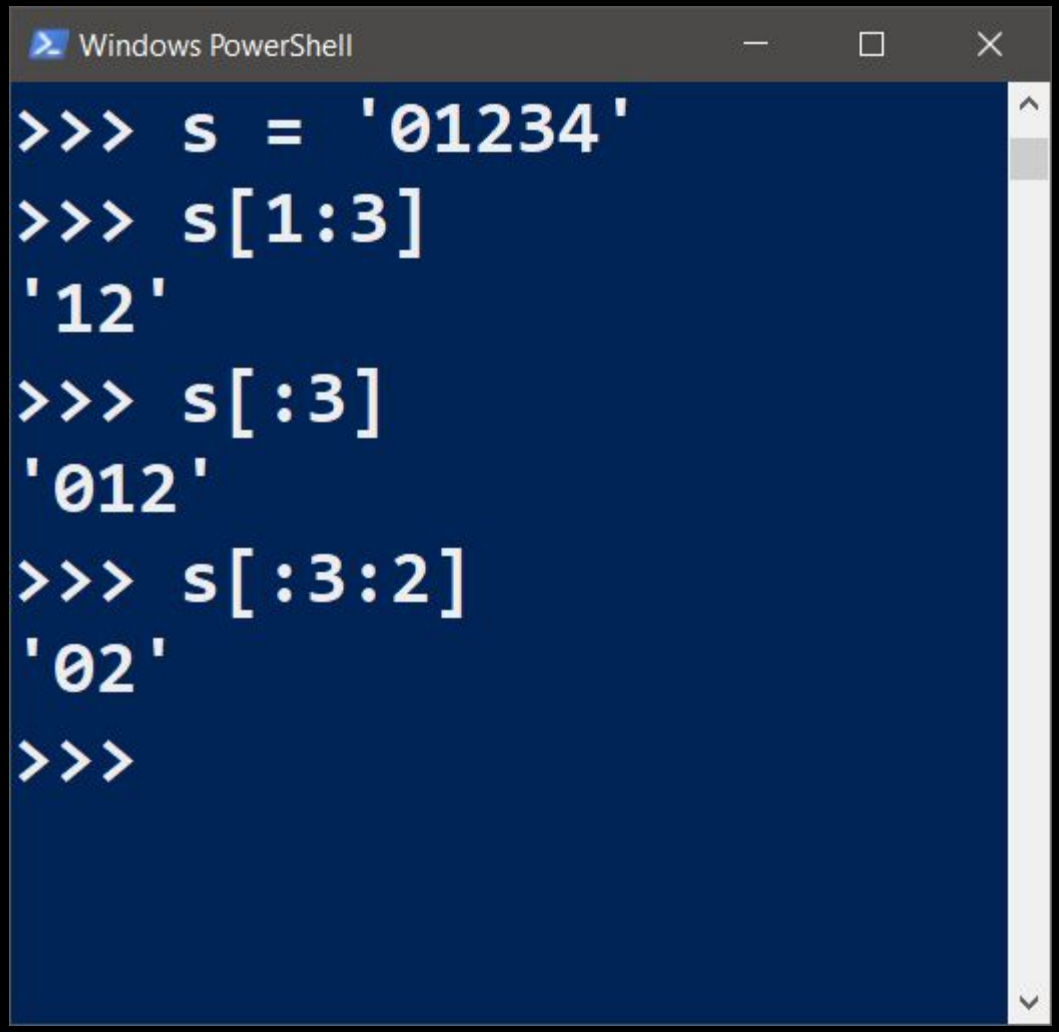
Some Slices



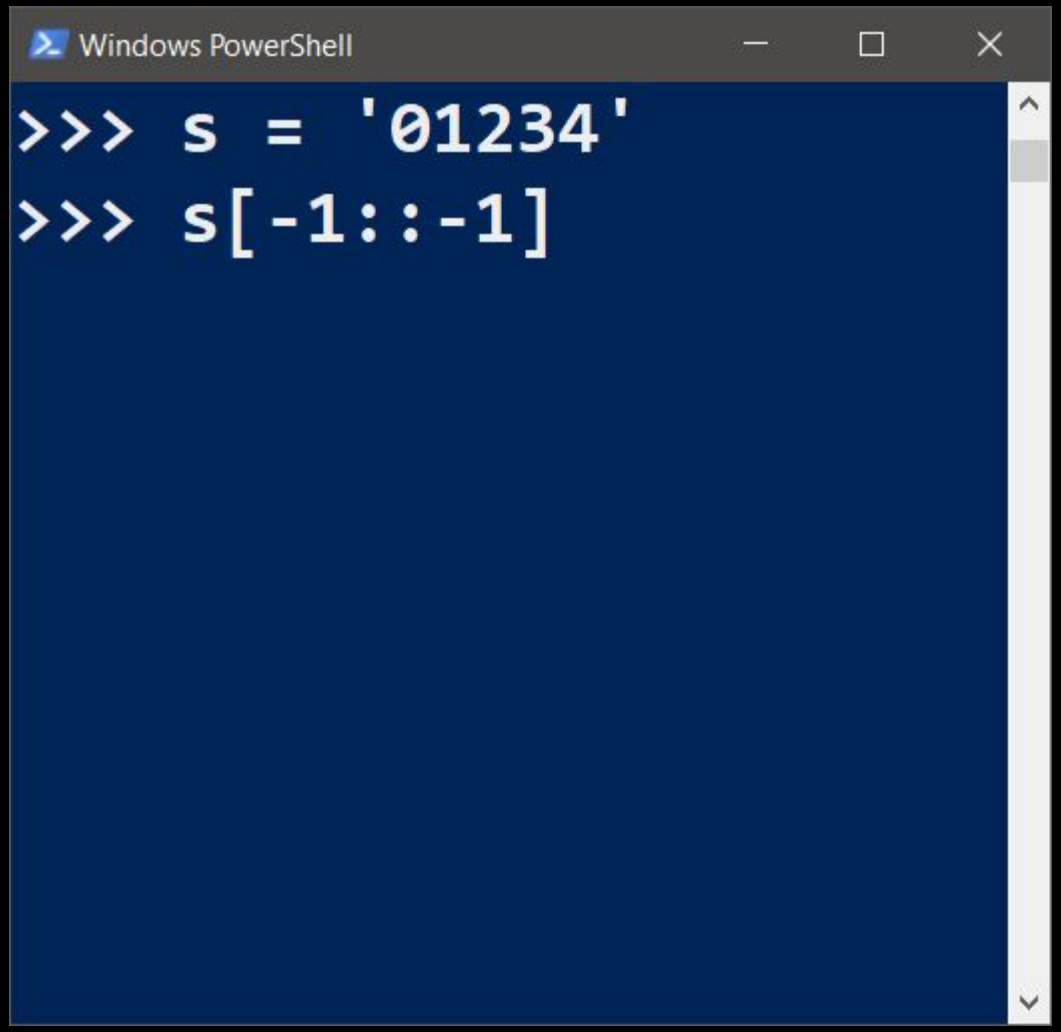
Some Slices



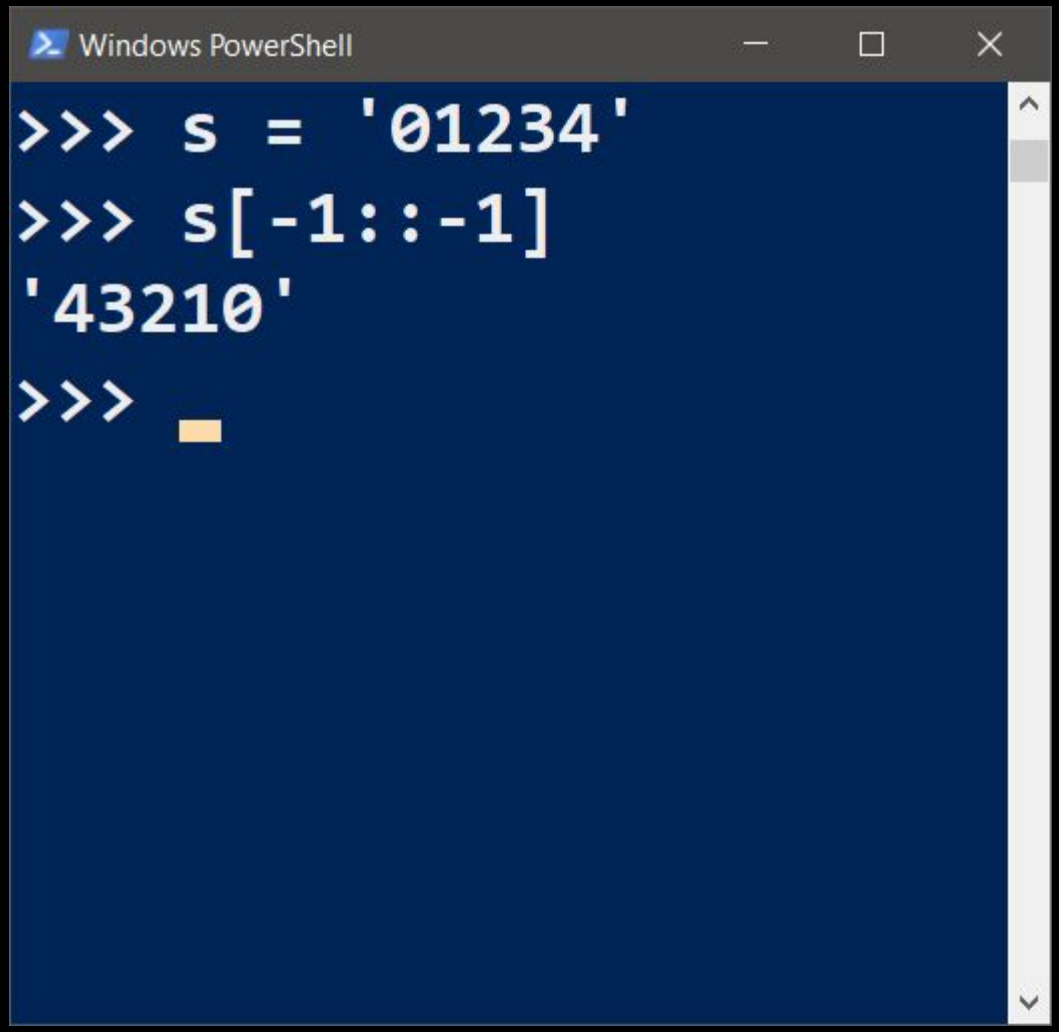
Some Slices



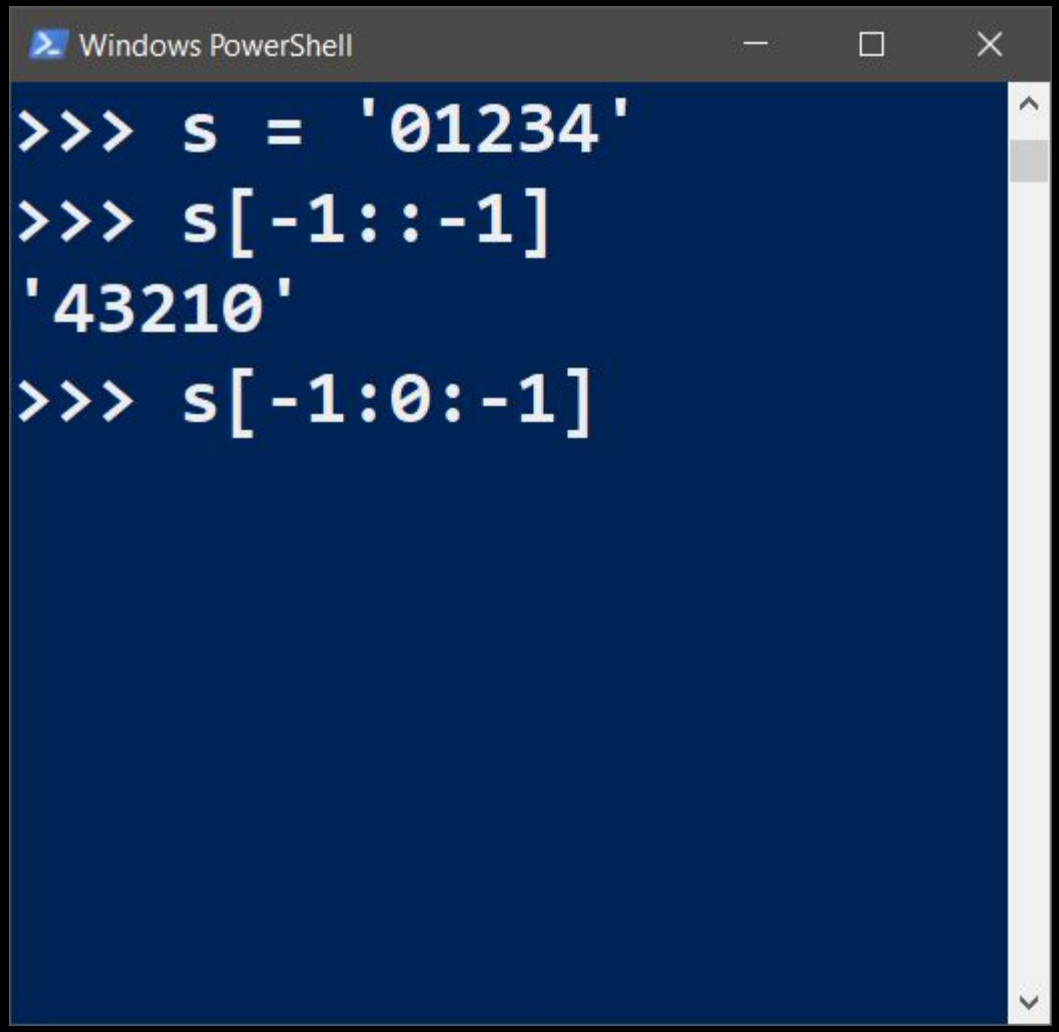
Some Slices



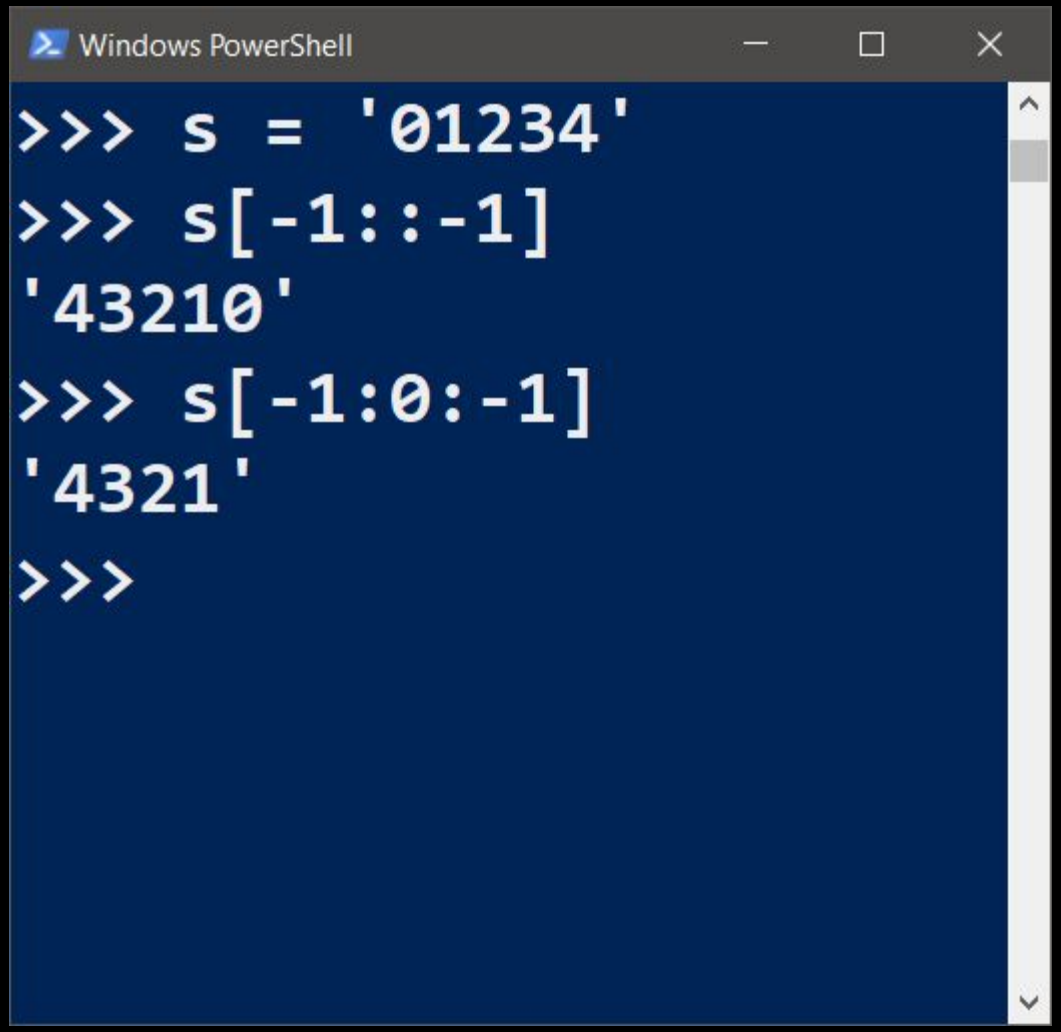
Some Slices



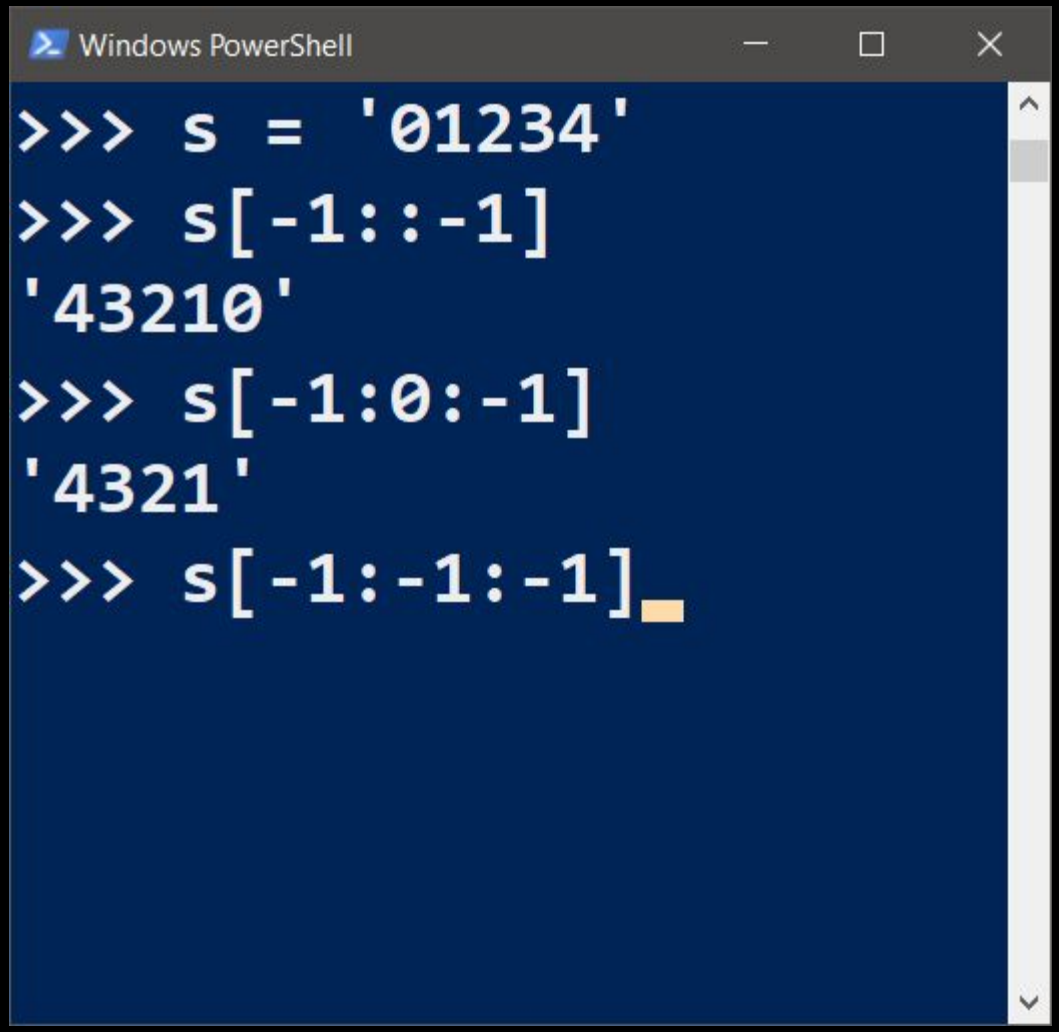
Some Slices



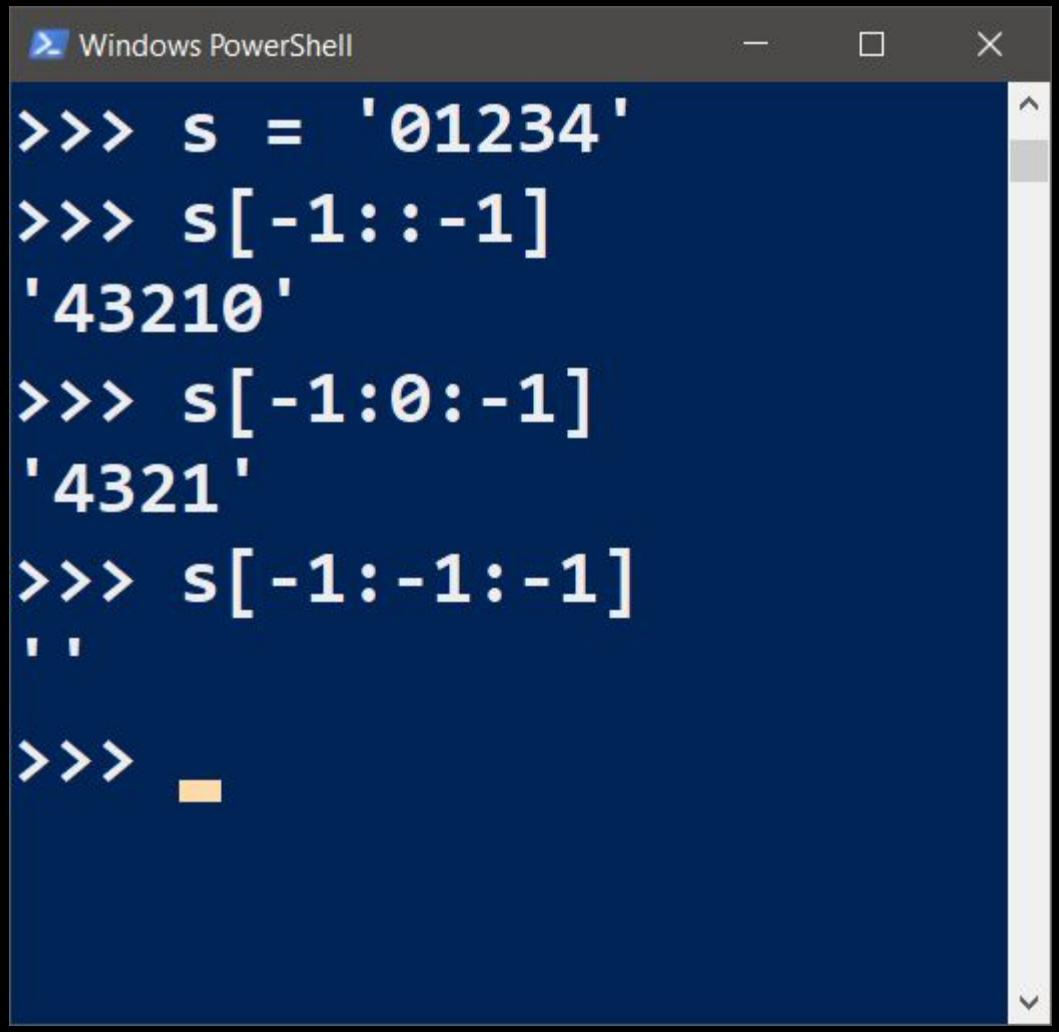
Some Slices



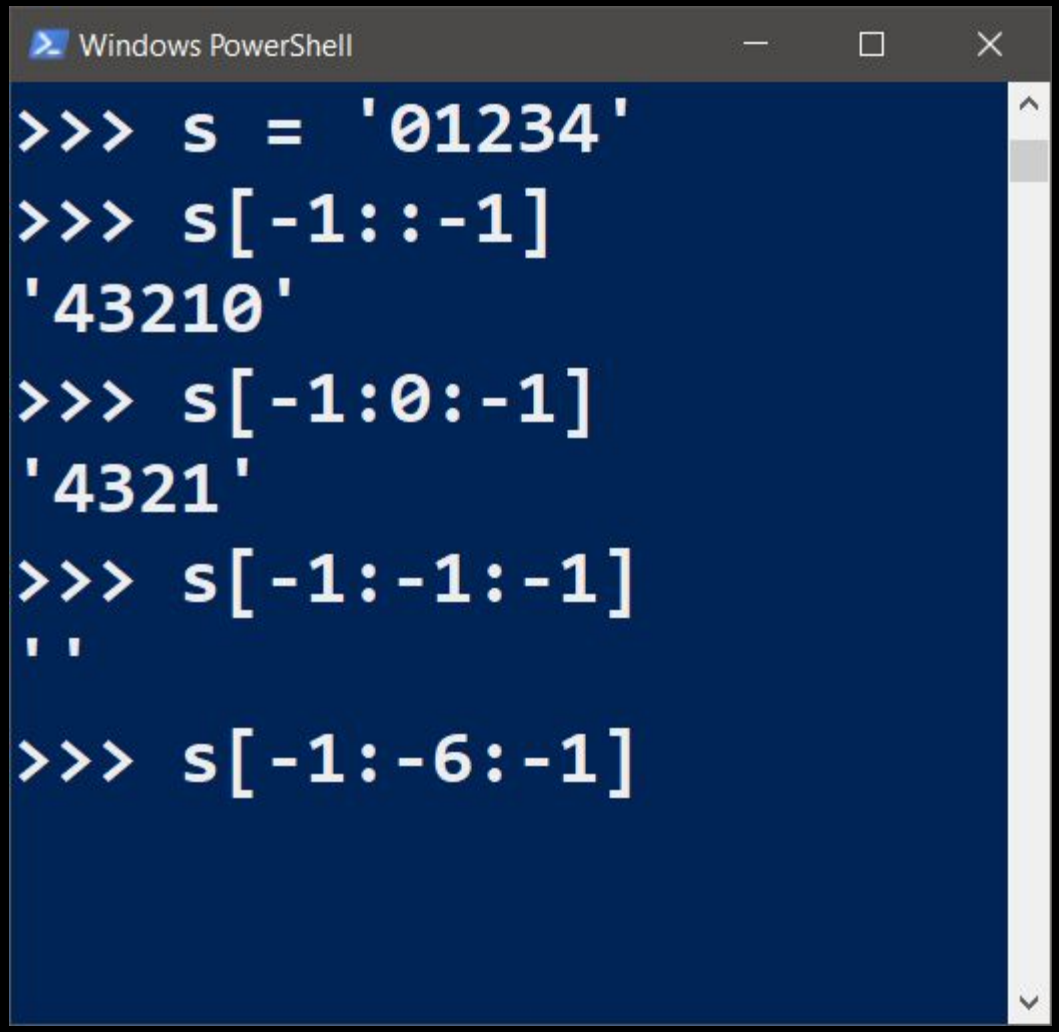
Some Slices



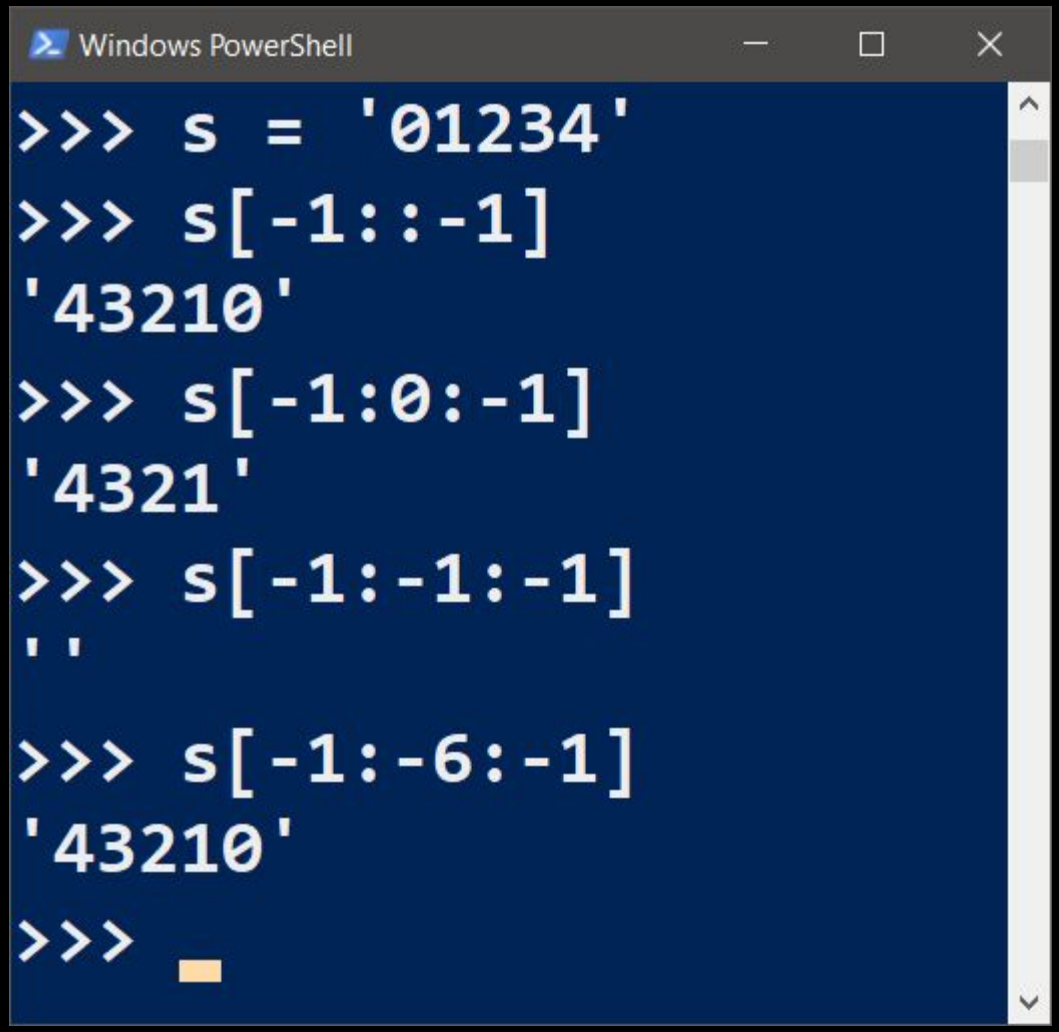
Some Slices



Some Slices



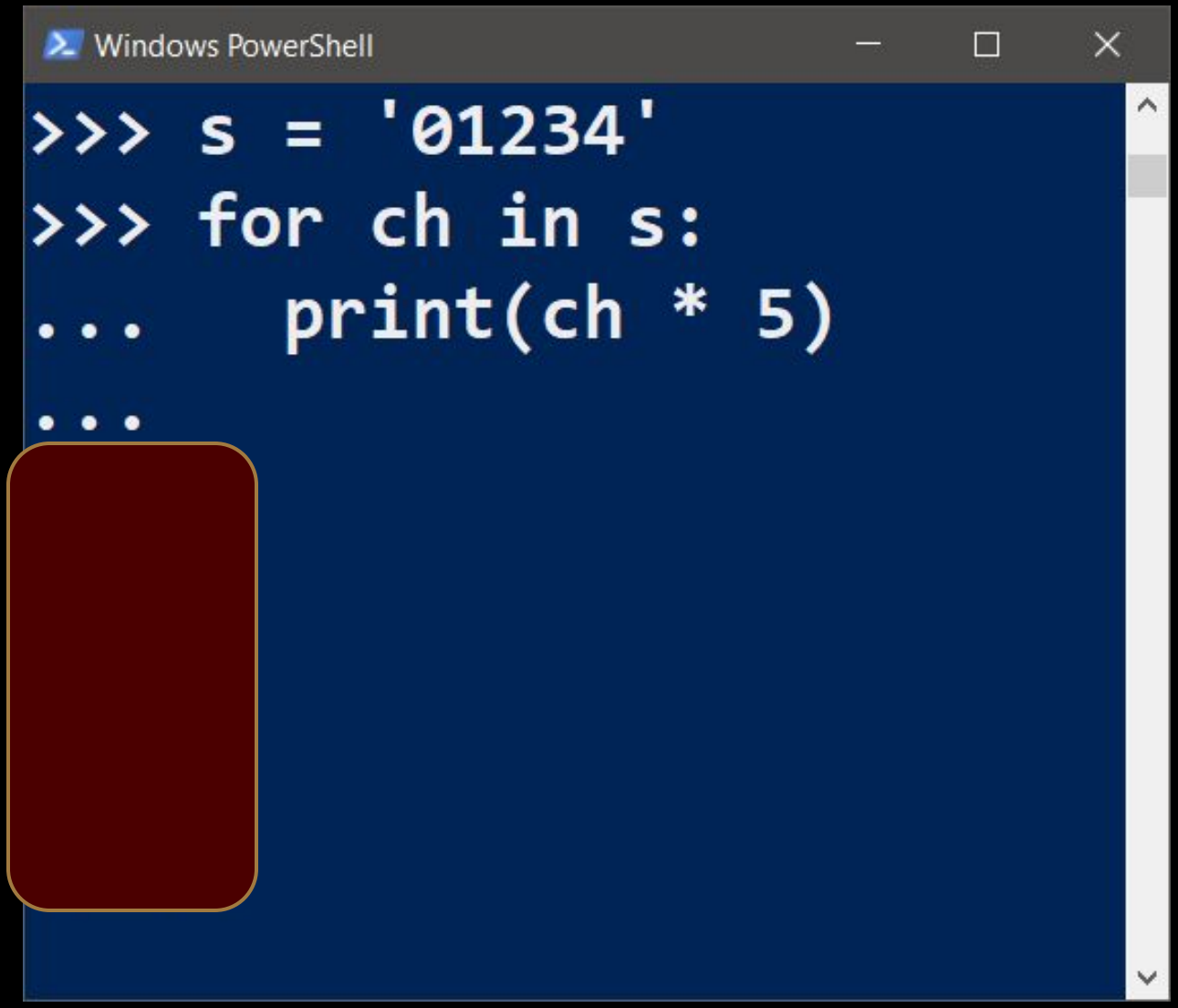
Some Slices


Windows PowerShell
>>> s = '01234'
>>> s[-1::-1]
'43210'
>>> s[-1:0:-1]
'4321'
>>> s[-1:-1:-1]
''
>>> s[-1:-6:-1]
'43210'
>>>

Slice vs. `range`

- **Don't get confused between slice indices and `range` arguments.**
- **Slices can use negative indices to access elements of the sequence.**
- **`range` just generates integers, which can be negative.**

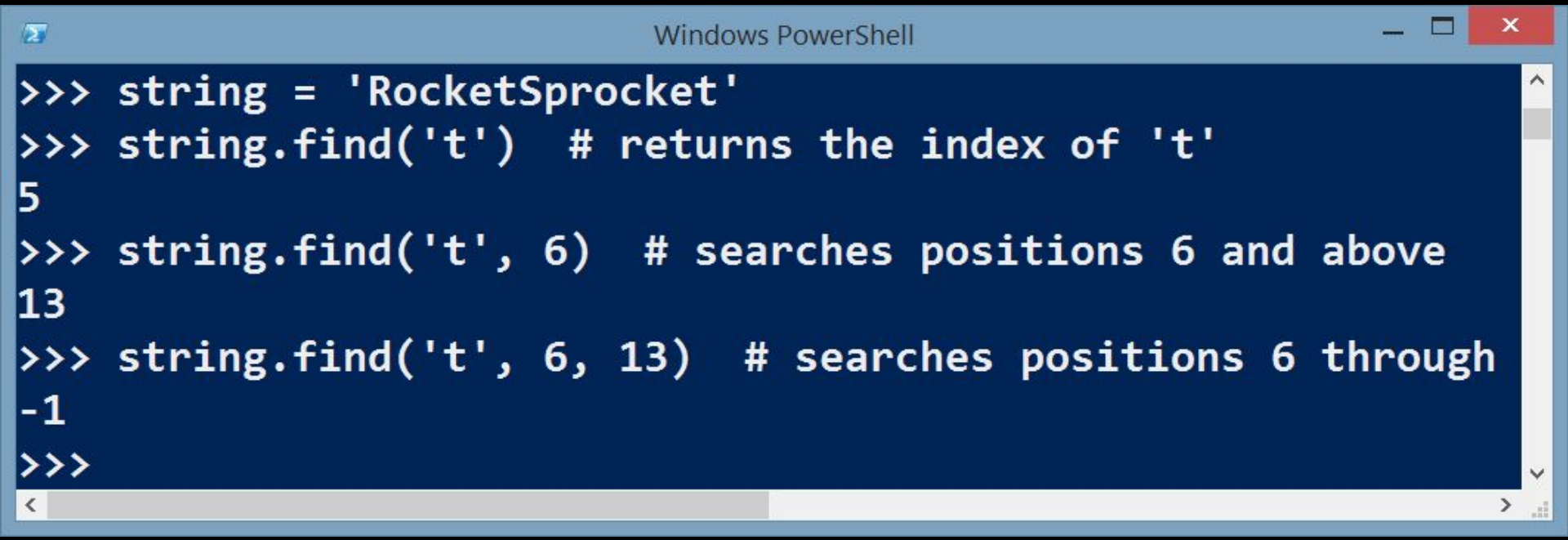
Traverse by Element



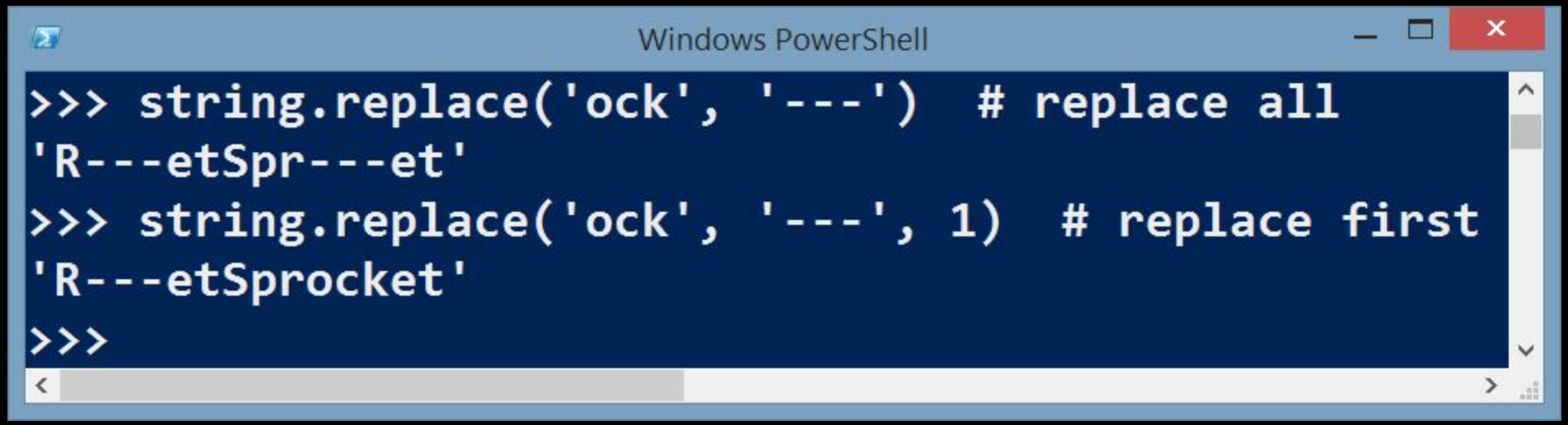
String Methods

- **Work just like turtle methods.**
- **Call them on a string literal or variable using the dot operator.**

find


Windows PowerShell
>>> string = 'RocketSprocket'
>>> string.find('t') # returns the index of 't'
5
>>> string.find('t', 6) # searches positions 6 and above
13
>>> string.find('t', 6, 13) # searches positions 6 through
-1
>>>

replace


Windows PowerShell
>>> string.replace('ock', '---') # replace all
'R---etSpr---et'
>>> string.replace('ock', '---', 1) # replace first
'R---etSprocket'
>>>

Files

- Up to now all of our data has been either *hard-coded* (written into our programs), or entered by the user while a program runs.
- We need a way to get our hands on much larger data sets
- Our programs can read large amounts of data from files

Files

- When we humans want to look at a file we:
  - open the file (e.g. double-click it)
  - read it (with our eyes)
  - close the file (e.g. click the X)
- Our `Python` programs will also:
  - open files
  - read them
  - and close them

Opening and Reading Files

- The built-in **open** function opens a file:

```
open(<filename>, <mode>)
```

- `open` returns a file *object* that we use to read the file:

```
# open a file in 'read' mode and
# store the file object in a variable:
file_reader = open('somefile.txt', 'r')
```

- The `close` method tells a file object to release the file once we're done with it:

```
file_reader.close()
```

Reading Files with a `For` Loop

- Here's how we read a file line by line with a for-loop:

```
file_reader = open('example0.txt', 'r')

for line in file_reader:
    print(line)
    print('repr: ' + repr(line))
```

- What does this `for` loop remind you of?
  - it's similar to looping through every character in a string
- The for-loop iterates through each line in the file
  - On each iteration `Python` assigns a string to `line`
  - The string is the next single line of text from the file
  - When there are no more lines left the loop exits

We call this traversing a file by line.

Reading Files with a `For` Loop

Contents

```
hello
```

- Let's have our program examine each line:

```
file = open('example0.txt', 'r')

for line in file:
    print('Length of line: ' + str(len(line)))
    print('Line: \"' + line + '\"')

file.close()
```

Reading Files with a `For` Loop

- Let's add a line to the text file and examine each line.

Contents

```
hello
goodbye
```

```
file = open('example1.txt', 'r')

for line in file:
    print('Length of line: ' + str(len(line)))
    print('Line: \"' + line + '\"')

file.close()
```

- The first string Python reads is `'hello\n'`
  - which has 6 characters

Reading Files with a `For` Loop

- Let's add a newline after goodbye.

Contents

```
hello
goodbye
```

```
file = open('example2.txt', 'r')

for line in file:
    print('Length of line: ' + str(len(line)))
    print('Line: \"' + line + '\"')

file.close()
```

- The newline character at the end of a line is part of the string.

"r" mode Error: No such file or directory

What happens if you give the wrong filename?

- If Python can't find the file you name you'll see something like this:

```
Traceback (most recent call last):
  File "filereading2.py", line 17, in <module>
    main()
  File "filereading2.py", line 8, in main
    file = open('exomple2.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory:
'exomple2.txt'
```

- The open function looks for the requested file in the same directory that the program file is in
  - It's easiest to just put the text file in the same directory as the .py file
  - If Python can't find it you either gave the wrong name, or the file is somewhere else.

Reading Files

Let's read a longer file and print out the contents:

First few lines of "iwalktheline.txt":

```
I Walk The Line, Johnny Cash

I keep a close watch on this heart of mine
I keep my eyes wide open all the time
I keep the ends out for the tie that binds
Because you're mine, I walk the line
```

...

```
file = open('iwalktheline.txt', 'r')
for line in file:
    print(line)
file.close()
```

- Why do we get a blank line between each line of text?

Remove the Newline

- The `print` function puts a newline (`'\n'`) at the end of each line it prints
- Each line in a text file is already ended with a newline.
- How can we remove the extra newline?
  - Use the `strip` method on each line we read in:

```
file = open('iwalktheline.txt', 'r')
for line in file:
    print(line.strip())
file.close()
```

Writing Files

We can also create and write to files:

- The second parameter to `open` represents the opening *mode* ("r" stands for "read").
- To `open` a file for writing we pass "w" for the mode:

```
filename = 'test.txt'
file_writer = open(filename, 'w')
```

Writing Files

- Write a string to a file:

```
filename = 'test.txt'
file_writer = open(filename, 'w')
file_writer.write('Hello new little file.')
file_writer.close()
```

- Run this and then verify that a new file has been created with the correct contents.

`write` **vs.** `print`

	`write`	`print`
Is a	method	function
Number of arguments	1	0 to infinity
Adds newline	no	yes



No commas separating arguments, have to use concatenation to make one argument only

Writing Files

- Let's run the same code again:

```
filename = 'test.txt'
file_writer = open(filename, 'w')
file_writer.write('Hello new little file.')
file_writer.close()
```

- Take a look at the file
- Now run this code:

```
filename = 'test.txt'
file_writer = open(filename, 'w')
file_writer.write("Hello again new little file.")
file_writer.close()
```

- Look at the file again

Writing Files

- So this:

```
file_writer = open("fileName.txt", "w")
```

- either **creates** a new file or **overwrites** an existing file:
  - if the named file *does not exist*, `open` creates a new file
  - if the named file *does exist*, `open` *overwrites* it

Writing Files

- Try running this:

```
filename = 'test.txt'
file_writer = open(filename, 'w')
file_writer.write('Hello new little file.')
file_writer.close()

file_writer = open(filename, 'w')
file_writer.close()
```

- Look at the file again
  - all gone :(

Writing Files

Now watch this:

```
filename = 'allOfMyMostImportantStuffInTheWorld.txt'
file_writer = open(filename, 'w')
file_writer.write('Guess what?')
file_writer.close()
```

- Don't worry, you would have to put your important file in the same directory as your program and open that file by name in your program. Just don't do that and you'll be fine.

Appending to a File

- If you want to add something to a file rather than overwriting it, open the file in "append" mode ("a"):

```
filename = 'test.txt'
file_writer = open(filename, 'w')
file_writer.write('Hello new little file.')
file_writer.close()

file_writer = open(filename, 'a')
file_writer.write('Hello again new little file.')
file_writer.close()
```

- Look at the file again
  - everything is there!

Writing Newlines

- Notice that so far all of our writes are on the same line in the file
- When writing to a file you have to explicitly add newlines where you want them (`'\n'`):

```
filename = 'test.txt'
file_writer = open(filename, 'w')
file_writer.write('Hello new little file.\n')
file_writer.write('Hello again new little file.\n')
file_writer.close()
```

- Look at the file again
  - we now have newlines

Closing Files After Use

- Try running this:

```
writer = open(filename, 'w')
writer.write('Hello new little file.\n')
writer.write('How are you today?\n')
name = input('Enter your name: ')
writer.write('\n\tYours truly,')
writer.write('\n\t' + name)

writer.close() # comment this out and re-run

reader = open(filename, 'r')
for line in reader:
    print('Line:\t' + line.rstrip())
reader.close()
```

Closing Files After Use

- So, `Python` might wait until the last minute to actually write things out to your file
- Closing your file causes `Python` to go ahead and write everything out

```
''' helpful string methods and operators '''
# isdigit()
# find()
# capitalize()
# replace()
# strip()
# in

''' helpful file methods '''
# write()

''' example of traversing a file by line '''
file = open(fname, 'r')
for line in file:
    # do something
file.close()
```

Worksheet – remember your string toolbox: concatenation, repetition, indexing, slicing, methods

A slice: `'rocket'[2:4]`